

Comprehensive Software Testing of SafeVoice- Women Safety Issue Reporting System

Introduction :

SafeVoice is a web-based Women Safety Issue Reporting System designed to provide a secure and user-friendly platform for reporting, managing, and resolving safety-related issues. The application allows users to report incidents, administrators to assign resolvers, and resolvers to investigate and provide solutions. The SafeVoice application is fully deployed on the cloud (Render Platform) and accessible via a live URL, enabling real-world testing on a production-like environment. The system follows a role-based access approach involving User, Admin, and Resolver modules, ensuring structured workflow and accountability. To ensure the quality, reliability, performance, and usability of the application, multiple software testing techniques were applied systematically.

Objective :

The main objective of this project is to perform comprehensive software testing on the SafeVoice Women Safety Issue Reporting Web Application to ensure its quality, reliability, usability, and performance.

The specific objectives are:

- To understand and analyze the functional workflow of the SafeVoice application involving User, Admin, and Resolver roles.
- To perform Unit Testing to validate individual modules such as login, SignUp.
- To perform Integration Testing to ensure smooth interaction between different system modules.
- To conduct System Testing to verify complete end-to-end functionality in a real deployed environment.
- To perform UI Testing using Selenium WebDriver to verify navigation menus, forms, buttons, dropdowns, links, headings, and images.
- To perform Performance Testing using Apache JMeter to evaluate response time, throughput, and system stability under load.
- To verify data integrity, session handling, and issue lifecycle flow from reporting to resolution acknowledgment.
- To identify defects and improve overall software quality by validating system behavior under different conditions.
- To gain practical experience with industry-standard testing tools such as Selenium WebDriver, TestNG, Postman and Apache JMeter.

Used Deployed Website for Testing:

- **Application Name:** SafeVoice
- **Deployment Platform:** Render Cloud
- **Technology Stack:** HTML, EJS, CSS, JavaScript, Node.js, Express, PostgreSQL
- **Testing Tool Used:** Selenium WebDriver with TestNG, JMeter, Postman
- **Browser:** Mozilla Firefox

Testing was performed on the live deployed website, which helps validate the system in real-world conditions.

Types of Testing Performed :

The following testing types were performed on the SafeVoice application:

1. Unit Testing
2. Integration Testing
3. System Testing
4. Performance Testing
5. UI (User Interface) Testing
6. Acceptance Testing

Each testing type is explained below.

1) Unit Testing :

Unit Testing is a software testing technique in which individual modules or components of an application are tested independently to verify that each unit works correctly in isolation.

In the SafeVoice web application, Unit Testing was performed on the authentication modules, specifically:

- User Login
- User Signup
- Admin Login
- Admin Signup
- Resolver Login
- Resolver Signup

These modules were tested using API-based unit testing to validate request and response behavior.

Login and Signup modules were selected for unit testing because:

- They are independent and self-contained modules.
- They do not depend on other system components.
- They represent the entry point of the application.
- Any failure in authentication can block the entire system workflow.
- They can be easily validated by checking input fields, credentials, and responses.

The following operations were not included in unit testing:

- Issue Reporting
- Issue Assignment
- Solution Submission

Reason:

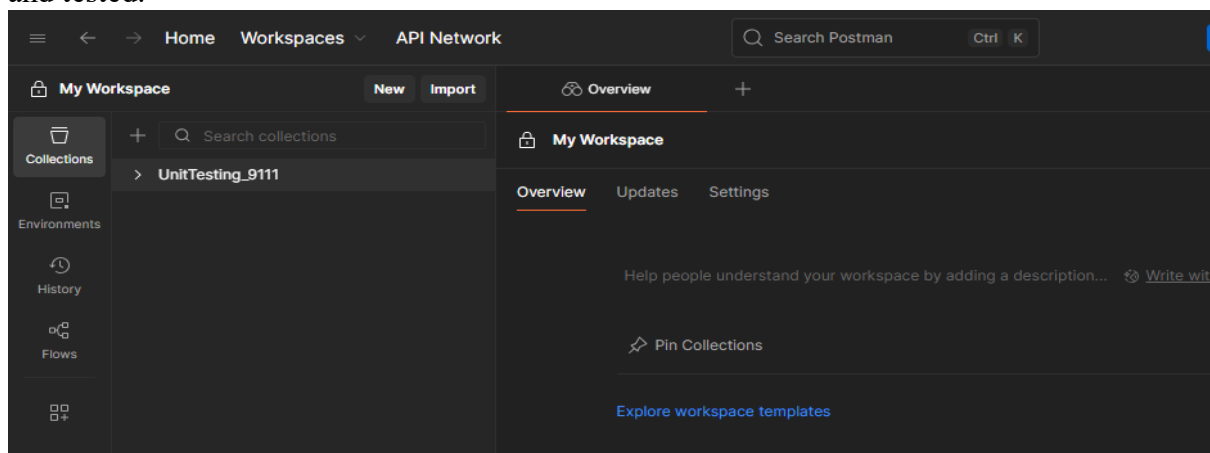
- These modules depend on multiple roles and modules (User, Admin, Resolver).
- They involve database relationships (joins), session handling, and role-based workflows.
- They cannot function independently and require module interaction.

Hence, these operations are more suitable for Integration Testing and System Testing rather than Unit Testing.

Tool Used for Unit Testing :-

- Postman
- HTTP methods: POST
- Validation of request parameters and response status

Performing Unit Testing :- Open Postman tool and Create a collections → Click on + icon and create UserSignUp Post request and Click on form-urlencoded and specify the form data that need to be pass for signup in key-value pair in table format , like this all other request are created and tested.



1)UNIT TEST-1: USER SIGNUP

Request Details

- **Method:** POST
- **URL:** <https://safevoice-webapp.onrender.com/signup/user>

Request Details:

- Method: POST
- URL: <https://safevoice-webapp.onrender.com/signup/user>
- Body: x-www-form-urlencoded

Response: 200 OK

Data Output:

	user_id [PK] integer	name character varying (100)	email character varying (100)	phone character varying (15)	password character varying (100)
1	1	Jagruti	J@gmail.com	98765434	1234
2	2	Riya Shinde	R@gmail.com	9876545	riu89
3	3	testuser	test@gmail.com	987654321	test123

2)UNIT TEST-2: USER LOGIN (with wrong password and right email)

Request Details

- Method: POST
- URL: <https://safevoice-webapp.onrender.com/login/user>

The screenshot shows a web browser interface with the URL <https://safevoice-webapp.onrender.com/login/user>. The page has a dark purple header with a 'Back to Home' link and a red error message: 'Invalid email or password'. Below the error message are two buttons: 'User Login' and 'User Signup'.

3)UNIT TEST-3: USER LOGIN (with right password and right email)

Request Details

- Method: POST
- URL: <https://safevoice-webapp.onrender.com/login/user>

The screenshot shows a web browser interface with the URL <https://safevoice-webapp.onrender.com/login/user>. The page has a dark purple header with the 'SafeVoice' logo and navigation links: 'Home', 'Report Issue', 'My Issues', and 'Logout'. Below the navigation bar is a large white box with the text 'Welcome to SafeVoice'.

4)UNIT TEST-4: Admin SignUp :

Request Details

- Method: POST
- URL: <https://safevoice-webapp.onrender.com/signup/admin>

The screenshot displays a REST client interface for a POST request to `https://safevoice-webapp.onrender.com/signup/admin`. The request body is `x-www-form-urlencoded` and contains the following data:

Key	Value	Description
name	admin12	
email	ad@gmail.com	
phone	898765432	
password	78651	

The response is a `200 OK` status with a response time of 317 ms and a size of 1.42 KB. Below the response, a web application interface is shown with a success message: "Admin registered successfully". The interface includes two sections: "Admin Login" and "Admin Signup".

Admin Login

Email
Password
Login

Admin Signup

Name
Email
Phone

Data Output

	admin_Id [PK] integer	name character varying (100)	email character varying (100)	password character varying (100)
1	1	Janvi Agarwal	Ja@gmail.com	345
2	2	admin12	ad@gmail.com	78651

5)UNIT TEST-5: Admin Login :

Request Details

- Method: POST
- URL: <https://safevoice-webapp.onrender.com/login/admin>

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** <https://safevoice-webapp.onrender.com/login/admin>
- Body:** x-www-form-urlencoded
- Form Data:**

Key	Value	Description
email	ad@gmail.com	
password	78651	
- Status:** 200 OK
- Response:** The response is an HTML page titled "SafeVoice - Admin". It features a navigation bar with links: Home, View Issues, Assign Resolver, and Logout. Below the navigation bar is a large grey box labeled "Admin Dashboard".

6)UNIT TEST-6: Admin Login (With wrong email and wrong password)

Request Details

- Method: POST
- URL: <https://safevoice-webapp.onrender.com/login/admin>

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** <https://safevoice-webapp.onrender.com/login/admin>
- Body:** x-www-form-urlencoded
- Form Data:**

Key	Value	Description
email	ad@gmail.com	
password	786	
- Status:** 200 OK
- Response:** The response is an HTML page showing an error. It has a "Back to Home" link and a red alert box that says "Invalid admin credentials". Below this, there are two sections: "Admin Login" with an "Email" input field, and "Admin Signup" with a "Name" input field.

7)UNIT TEST-7: Resolver SignUp :

Request Details

- Method: POST
- URL: <https://safevoice-webapp.onrender.com/signup/resolver>

https://safevoice-webapp.onrender.com/signup/resolver

POST https://safevoice-webapp.onrender.com/signup/resolver

Docs Params Authorization Headers (10) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL

	Key	Value	Description
☒	name	Priyank sharma	
☒	email	p@gmail.com	
☒	role	lawyer	
☒	phone	98765456	
☒	password	9768	

Overview POST https://safevoice-web. No environment

https://safevoice-webapp.onrender.com/signup/resolver

POST https://safevoice-webapp.onrender.com/signup/resolver

Docs Params Authorization Headers (10) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL

Body Cookies (1) Headers (14) Test Results 200 OK 332 ms 1.51 KB

HTML Preview Visualize

← Back to Home Resolver registered successfully

Resolver Login

Resolver Signup

Data Output Messages Notifications						
	resolver_id [PK] integer	name character varying (100)	role character varying (50)	phone character varying (15)	email character varying (100)	password character varying (255)
1	1	Jiya Rathod	Legal Advisor	9876543254	Ji@gmail.com	789
2	2	Priyank sharma	lawyer	98765456	p@gmail.com	9768

8)UNIT TEST-8: Resolver login (with right email and right password)

Request Details

- Method: POST
- URL: <https://safevoice-webapp.onrender.com/login/resolver>

The screenshot shows a REST client interface with a POST request to `https://safevoice-webapp.onrender.com/login/resolver`. The request body is `x-www-form-urlencoded` with the following data:

Key	Value	Description
email	p@gmail.com	
password	9768	

The response is a 200 OK status with a response time of 343 ms and a body size of 1.17 KB. The response body is an HTML page titled "SafeVoice - Resolver" with a header "Assigned Issues" and a message "No issues assigned yet."

9)UNIT TEST-9 : Negative Test Case Tried (User SignUp)

The backend does not include JSON parsing middleware, so JSON requests are not processed. The application is designed to accept form-urlencoded data submitted by HTML forms. So User SignUp failed.

The screenshot shows a REST client interface with a POST request to `https://safevoice-webapp.onrender.com/signup/user`. The request body is `raw` (JSON) with the following data:

```
1 {
2   "name": "Test User",
3   "email": "testuser@gmail.com",
4   "phone": "9876543210",
5   "password": "test123"
6 }
7
```

The response is a 500 Internal Server Error with a response time of 1.16 s and a body size of 811 B. The response body is a JSON object with the following error message:

```
TypeError: Cannot destructure property 'name' of 'req.body' as it is undefined.
    at /app/app.js:55:11
    at Layer.handleRequest (/app/node_modules/router/lib/layer.js:152:17)
    at next (/app/node_modules/router/lib/route.js:157:13)
    at Route.dispatch (/app/node_modules/router/lib/route.js:117:3)
    at handle (/app/node_modules/router/index.js:435:11)
    at Layer.handleRequest (/app/node_modules/router/lib/layer.js:152:17)
    at /app/node_modules/router/index.js:295:15
    at processParams (/app/node_modules/router/index.js:582:12)
    at next (/app/node_modules/router/index.js:291:5)
    at session (/app/node_modules/express-session/index.js:487:7)
```

Test Case ID	Test Case Description	Method	Endpoint (URL)	Expected Result
UNIT TEST-1	User Signup (Valid Data)	POST	/signup/user	User account created successfully
UNIT TEST-2	User Login (Correct Email, Wrong Password)	POST	/login/user	Login failed with error message
UNIT TEST-3	User Login (Correct Email & Password)	POST	/login/user	User logged in successfully
UNIT TEST-4	Admin Signup (Valid Data)	POST	/signup/admin	Admin account created successfully
UNIT TEST-5	Admin Login (Valid Credentials)	POST	/login/admin	Admin logged in successfully
UNIT TEST-6	Admin Login (Wrong Email & Password)	POST	/login/admin	Login failed with error message
UNIT TEST-7	Resolver Signup (Valid Data)	POST	/signup/resolver	Resolver account created successfully
UNIT TEST-8	Resolver Login (Valid Credentials)	POST	/login/resolver	Resolver logged in successfully
UNIT TEST-9	User Signup (Negative Test – JSON Data)	POST	/signup/user	Signup failed due to unsupported JSON format

Unit testing successfully verified that all authentication modules of the SafeVoice application function correctly in isolation. This ensured a stable foundation for further testing phases such as Integration, System, UI, and Performance Testing.

2) Integration Testing :

Integration Testing verifies that two or more modules work together correctly after being combined.

Instead of testing individual features separately, it checks the data flow and interaction between modules.

In SafeVoice, different modules like Login, Issue Reporting, Admin Assignment, and Resolver Solution are interconnected, so testing their integration is necessary.

These tests are considered Integration Testing because:

- Each test involves multiple modules
- Output of one module becomes input for the next
- Real user roles interact with the same issue lifecycle
- Tested on a deployed application using automation tools

Tools Used

- Selenium WebDriver - for automated browser-based testing
- TestNG - for test execution, assertions, and reporting

IDE Used

- Eclipse IDE

Testing Approach

- Black-Box Testing
- Automation Testing
- Testing on Deployed Application

1) IT-01: User Login → Report Issue :

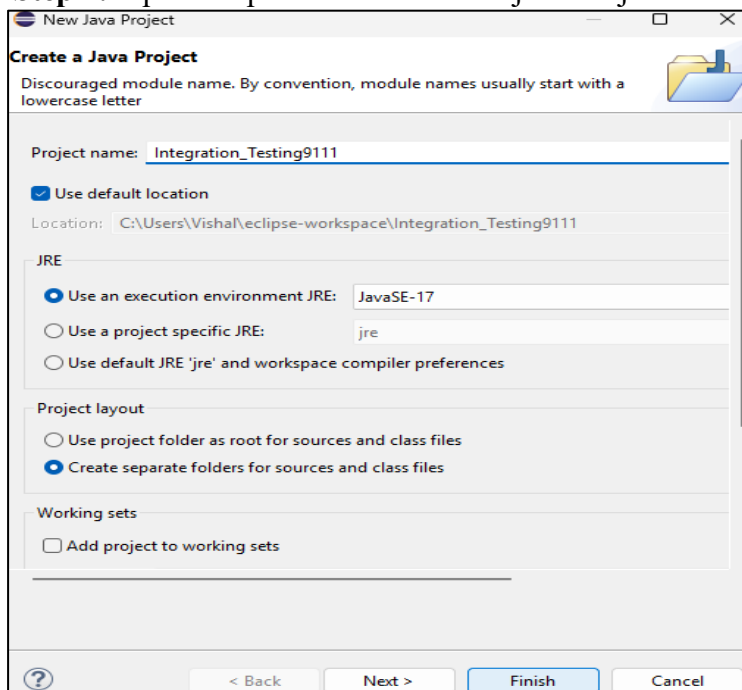
This test checks the integration between:

- User Authentication Module
- User Dashboard Module
- Issue Reporting Module

Here, successful login allows access to the report issue page, and the reported issue is stored for further processing. Since multiple modules work together, it is an integration test, not a unit test.

Steps1: Download selenium-server-standalone-3.141.59 jar and selenium-java-4.38 and download gecodriver . After installation make folder “myfolder” in that add all extracted selenium-java jar files and standalone jar, jcommander jar , testing jar , jxl jar and also add extracted gecodriver.exe file path inside java code.

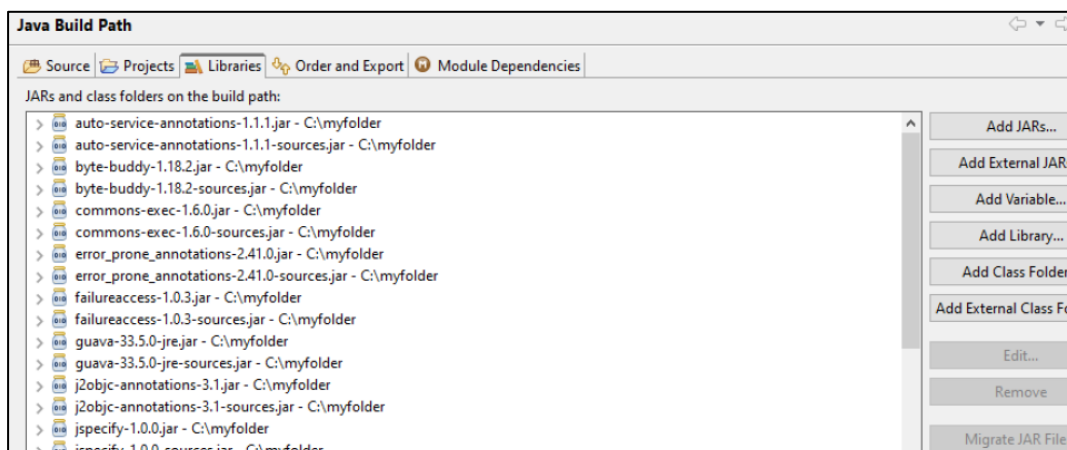
Step2 :-Open Eclipse → Create a new java Project → Name it → Click on Finish.



Name	Date modified
Last month	
jcommander-1.82.jar	22-12-2025 20:27
testng-6.7.jar	22-12-2025 19:47
jxl-2.6.12.jar	22-12-2025 18:23

Step3 :- Also make sure TestNG is installed if not → installed from Eclipse marketplace.

Right Click on project → click on build path → configure Build path → click on libraries → Add jar files from the my folder and click on apply and close.

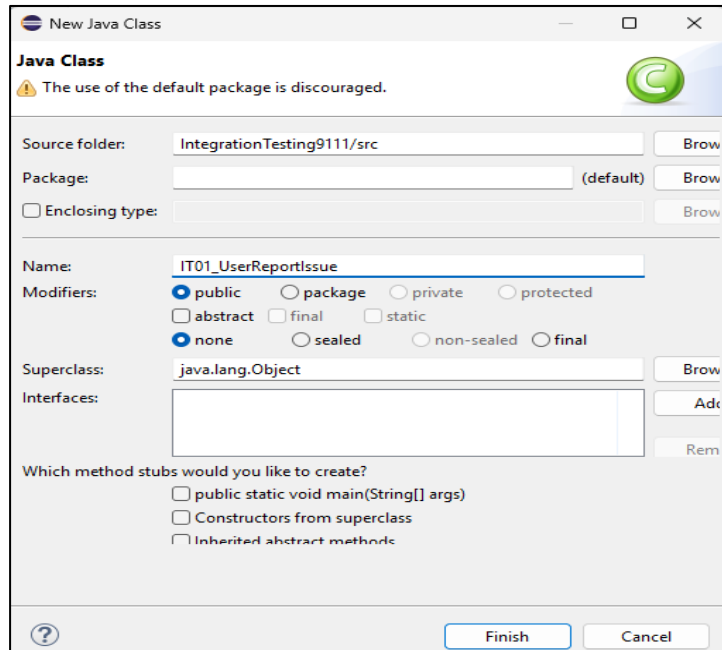


Step4 :

Create a java Class → Right click on project → new → class → name it → Finish.

Step5 :

Write code in the class and run the java class and perform the integration testing.



Code :-

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.support.ui.Select;
import org.testng.Assert;
import org.testng.annotations.*;
public class IT01_UserReportIssue {
    WebDriver driver;
    @BeforeTest
    public void setup() {
        System.setProperty("webdriver.chrome.driver", "D:\\myFolder\\geckodriver.exe");
        driver = new FirefoxDriver();
        driver.manage().window().maximize();
    }
    @Test
    public void userLoginAndReportIssue() throws InterruptedException {

        driver.get("https://safevoice-webapp.onrender.com/login/user");
        driver.findElement(By.name("email")).sendKeys("J@gmail.com");
        driver.findElement(By.name("password")).sendKeys("1234");
        driver.findElement(By.xpath("//button[contains(text(),'Login')]")).click();
        Thread.sleep(3000);

        driver.get("https://safevoice-webapp.onrender.com/dashboard/user");
        Thread.sleep(2000);
    }
}
```

```
driver.findElement(By.linkText("Report Issue")).click();
Thread.sleep(3000);

driver.findElement(By.name("title"))
    .sendKeys("Test Issue");

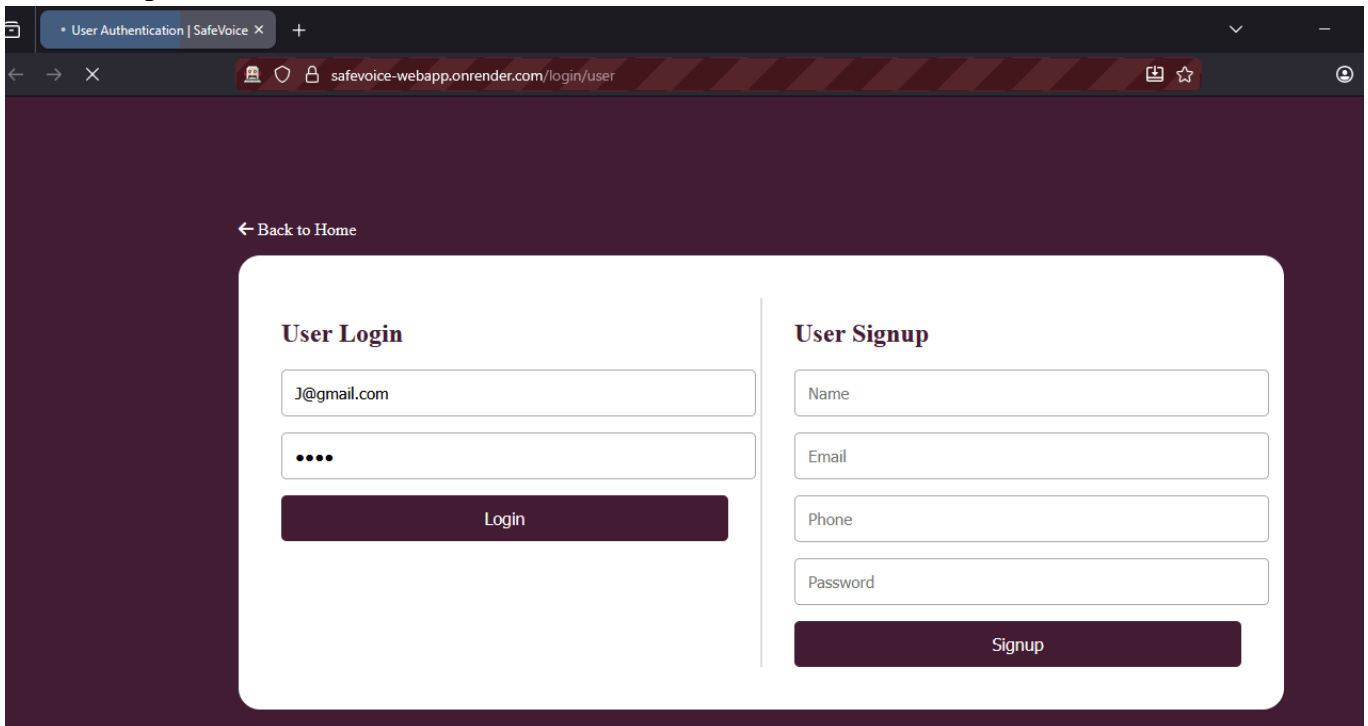
driver.findElement(By.name("description"))
    .sendKeys("Integration testing for user issue reporting");

Select category = new Select(
    driver.findElement(By.name("category")))
);
category.selectByVisibleText("Harassment");

driver.findElement(By.xpath("//button[contains(text(),'Submit')]"))
    .click();
Thread.sleep(4000);

Assert.assertTrue(driver.getCurrentUrl().contains("dashboard"));
System.out.println("Jagruti Tak 9111");
System.out.println("Integration Test Passed: User reported issue successfully");
}
@AfterTest
public void tearDown() {
    driver.quit();
}
}
```

Output :



The screenshot shows a web browser window with the address bar displaying "safevoice-webapp.onrender.com/login/user". The page has a dark purple background. At the top left, there is a link "← Back to Home". The main content area contains two white rounded rectangular boxes side-by-side. The left box is titled "User Login" and contains two input fields: the first has the text "J@gmail.com" and the second has four dots "....". Below these fields is a dark purple button labeled "Login". The right box is titled "User Signup" and contains four input fields labeled "Name", "Email", "Phone", and "Password". Below these fields is a dark purple button labeled "Signup".

User Dashboard | SafeVoice

safevoice-webapp.onrender.com/dashboard/user

SafeVoice

HomeReport IssueMy IssuesLogout

Report an Issue

Test Issue

Integration testing for user issue reporting

Harassment

Submit Issue

```
@ Javadoc Declaration Console
<terminated> IT01_UserReportIssue [TestNG] C:\Eclipse\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.10.v2024
[RemoteTestNG] detected TestNG version 6.7.0
[TestNG] Running:
C:\Users\Vishal\AppData\Local\Temp\testng-eclipse--1799930308\testng-customsuite.xml

Jagruti Tak 9111
Integration Test Passed: User reported issue successfully
PASSED: userLoginAndReportIssue

=====
Default test
Tests run: 1, Failures: 0, Skips: 0
=====

=====
Default suite
Total tests run: 1, Failures: 0, Skips: 0
=====

[TestNG] Time taken by [TestListenerAdapter] Passed:0 Failed:0 Skipped:0]: 0 ms
[TestNG] Time taken by org.testng.reporters.jq.Main@6c372fe6: 32 ms
[TestNG] Time taken by org.testng.reporters.JUnitReportReporter@6fa34d52: 6 ms
[TestNG] Time taken by org.testng.reporters.XMLReporter@1ef6d34c: 6 ms
[TestNG] Time taken by org.testng.reporters.EmailableReporter@5495333e: 1 ms
[TestNG] Time taken by org.testng.reporters.SuiteHTMLReporter@135606db: 24 ms
```

Problems Results of running class IT01_UserReportIssue

Search: Passed: 1 Failed: 0 Skipped: 0 Tests: 1/1 Methods: 1 (20529 ms)

All TestsFailed TestsSummary

Default suite (1/0/0/0) (16.597 s)

Default test (16.597 s)

IT01_UserReportIssue

userLoginAndReportIssue (16.597 s)

Failure Exception

Data Output

Messages

Notifications

Showing rows: 1 to 3

Page No: 1 of 1

	issue_id [PK] integer	user_id integer	title character varying (200)	description text
1	1	1	Domestic Abuse by Part...	I am facing physical and emotional abuse from my partner at h
2	2	1	Test Issue	Integration testing for user issue reporting
3	3	1	Test Issue	Integration testing for user issue reporting

2)IT-02: Admin Login → Assign Resolver

Modules Integrated

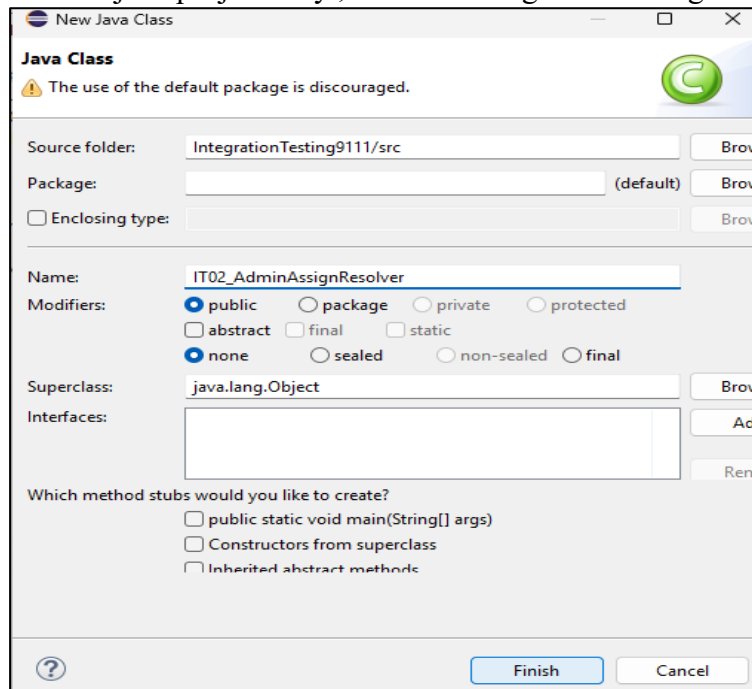
- Admin Authentication Module
- Issue Management Module
- Resolver Assignment Module

Reason for Integration Testing

This test validates that issues reported by users are correctly visible to the admin and can be assigned to a resolver.

It confirms the integration between user-generated data and admin actions.

In same java project only , add next Integration testing :



Code :

```
import org.openqa.selenium.*;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.firefox.FirefoxOptions;
import org.openqa.selenium.support.ui.*;
import org.testng.Assert;
import org.testng.annotations.*;
import java.time.Duration;
import java.util.List;
public class IT02_AdminAssignResolver {
    WebDriver driver;
    WebDriverWait wait;
    @BeforeTest
    public void setup() {
        System.setProperty("webdriver.gecko.driver", "D:\\myFolder\\geckodriver.exe");
        FirefoxOptions options = new FirefoxOptions();
        options.setBinary("C:\\Program Files\\Mozilla Firefox\\firefox.exe");
        driver = new FirefoxDriver(options);
        driver.manage().window().maximize();
    }
}
```

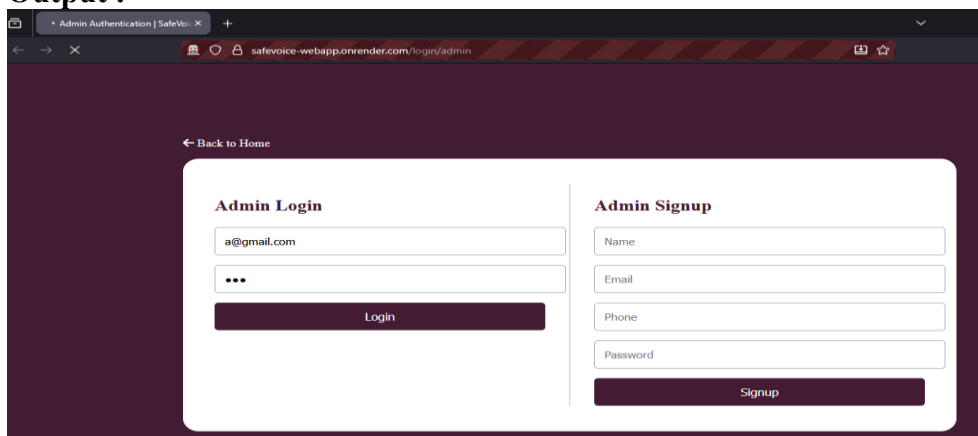
```
wait = new WebDriverWait(driver, Duration.ofSeconds(15));
}
@Test
public void adminAssignsIssueToResolver() throws InterruptedException {
    driver.get("https://safevoice-webapp.onrender.com/login/admin");
    driver.findElement(By.name("email")).sendKeys("a@gmail.com");
    driver.findElement(By.name("password")).sendKeys("421");
    driver.findElement(By.xpath("//button[contains(text(),'Login')]")).click();
    Thread.sleep(3000);
    driver.get("https://safevoice-webapp.onrender.com/dashboard/admin");
    Thread.sleep(3000);
    driver.findElement(By.linkText("Assign Resolver")).click();
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.xpath("//select")));

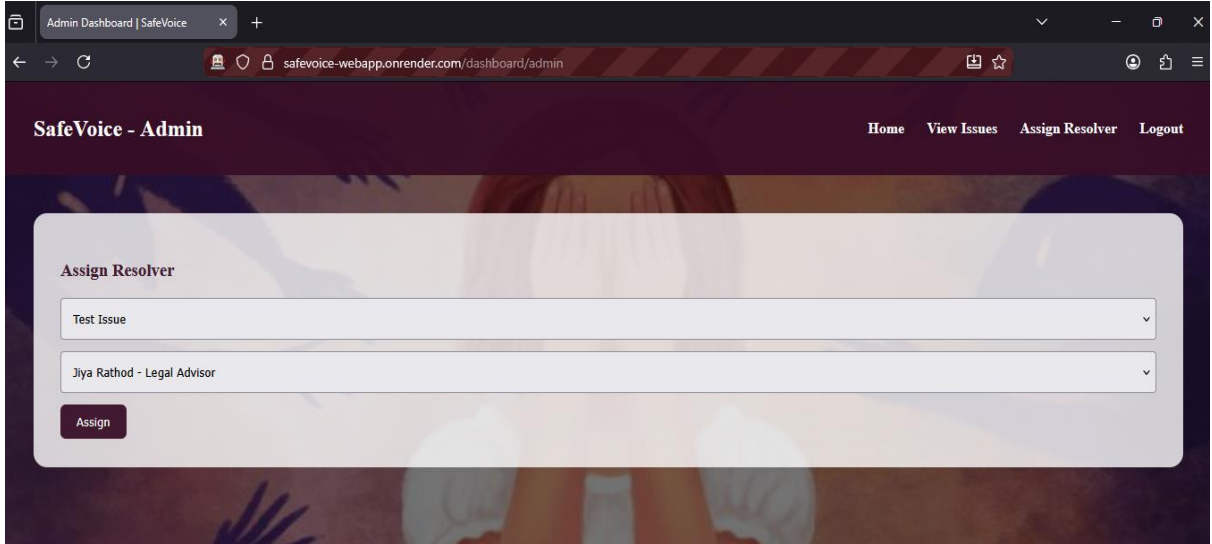
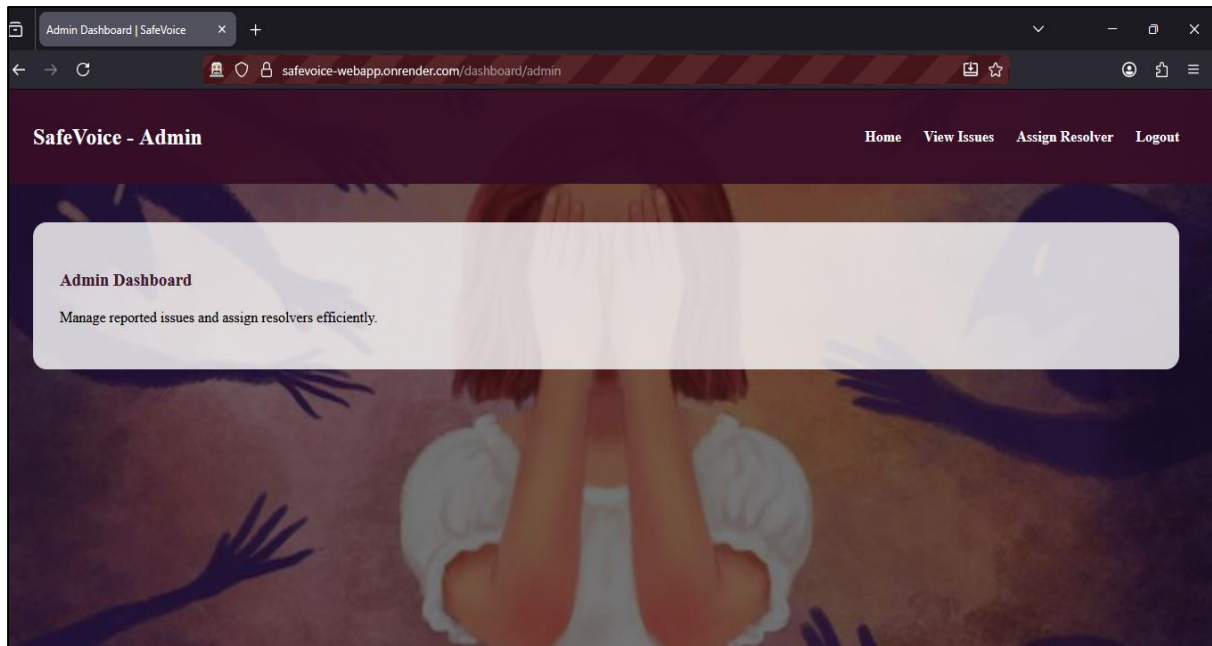
    List<WebElement> dropdowns = driver.findElements(By.xpath("//select"));
    Assert.assertEquals(dropdowns.size(), 2, "Two dropdowns must be present");
    Select issueDropdown = new Select(dropdowns.get(0));
    Assert.assertTrue(issueDropdown.getOptions().size() > 1);
    issueDropdown.selectByIndex(1);
    Thread.sleep(2000);
    Select resolverDropdown = new Select(dropdowns.get(1));
    Assert.assertTrue(resolverDropdown.getOptions().size() > 1);
    resolverDropdown.selectByIndex(1);
    Thread.sleep(2000);

    WebElement assignBtn = driver.findElement(By.xpath("//button[contains(text(),'Assign')]"));
    ((JavascriptExecutor) driver).executeScript("arguments[0].click();", assignBtn);
    Thread.sleep(4000);

    Assert.assertTrue(driver.getCurrentUrl().contains("admin"));
    System.out.println("Name : Jagruti Tak");
    System.out.println("Roll No : 9111");
    System.out.println("Integration Test Passed: Issue assigned successfully");
}
@AfterTest
public void tearDown() {driver.quit(); }
```

Output :





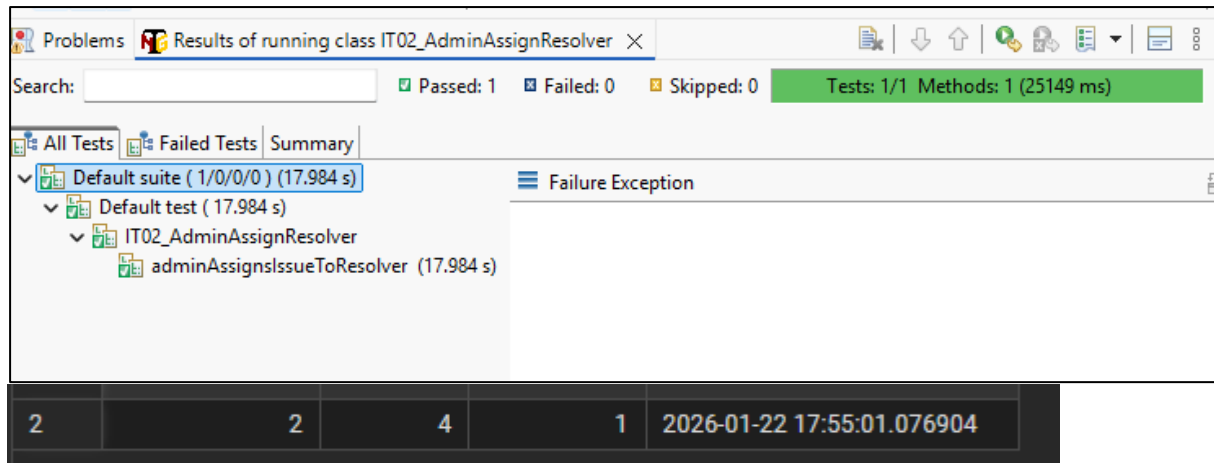
```
@ Javadoc Declaration Console X
<terminated> IT02_AdminAssignResolver [TestNG] C:\Eclipse\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64
[RemoteTestNG] detected TestNG version 6.7.0
[TestNG] Running:
  C:\Users\Vishal\AppData\Local\Temp\testng-eclipse-1525409511\testng-customsuite.xml

Name : Jagruti Tak
Roll No : 9111
Integration Test Passed: Issue assigned successfully
PASSED: adminAssignsIssueToResolver

=====
Default test
Tests run: 1, Failures: 0, Skips: 0
=====

Default suite
Total tests run: 1, Failures: 0, Skips: 0
=====

[TestNG] Time taken by [TestListenerAdapter] Passed:0 Failed:0 Skipped:0]: 0 ms
[TestNG] Time taken by org.testng.reporters.jq.Main@6c372fe6: 38 ms
[TestNG] Time taken by org.testng.reporters.JUnitReportReporter@6fa34d52: 9 ms
[TestNG] Time taken by org.testng.reporters.XMLReporter@1ef6d34c: 8 ms
[TestNG] Time taken by org.testng.reporters.EmailableReporter@5495333e: 2 ms
[TestNG] Time taken by org.testng.reporters.SuiteHTMLReporter@135606db: 23 ms
```



3) IT-03: Resolver Login → Submit Solution

Modules Integrated

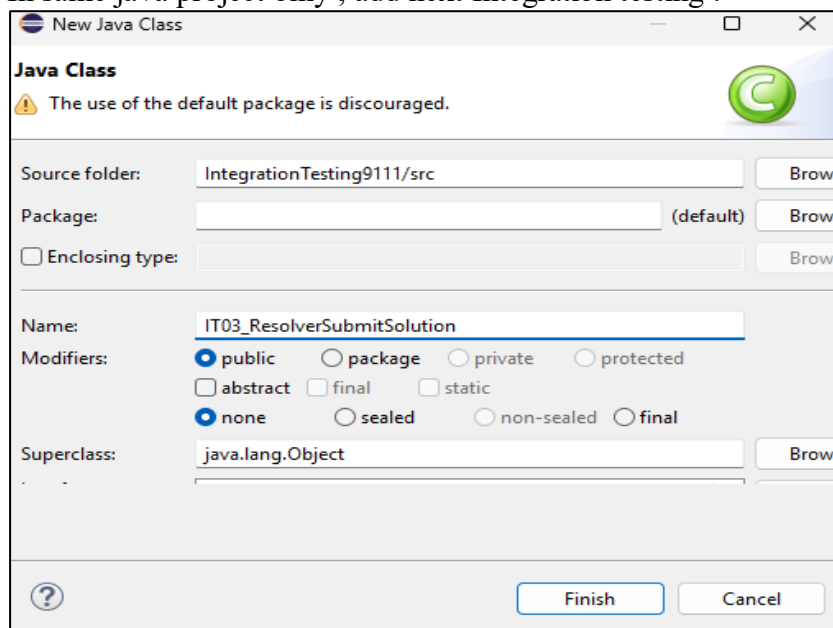
- Resolver Authentication Module
- Assigned Issues Module
- Solution Submission Module

Reason for Integration Testing

This test ensures that issues assigned by the admin are accessible to the resolver and that the resolver can submit solutions successfully.

This confirms smooth integration of admin and resolver modules.

In same java project only , add next Integration testing :



Code :

```
import org.openqa.selenium.*;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.firefox.FirefoxOptions;
import org.openqa.selenium.support.ui.*;
import org.testng.Assert; import org.testng.annotations.*;
import java.time.Duration;
import java.util.List;
```

```
public class IT03_ResolverSubmitSolution {
    WebDriver driver;
    WebDriverWait wait;
    @BeforeTest
    public void setup() {
        System.setProperty("webdriver.gecko.driver", "D:\\myFolder\\geckodriver.exe");
        FirefoxOptions options = new FirefoxOptions();
        options.setBinary("C:\\Program Files\\Mozilla Firefox\\firefox.exe");
        driver = new FirefoxDriver(options);
        driver.manage().window().maximize();
        wait = new WebDriverWait(driver, Duration.ofSeconds(15)); }

    @Test
    public void resolverSubmitsSolutionForLatestIssue() throws InterruptedException {

        driver.get("https://safevoice-webapp.onrender.com/login/resolver");
        driver.findElement(By.name("email")).sendKeys("Ji@gmail.com");
        driver.findElement(By.name("password")).sendKeys("789");
        driver.findElement(By.xpath("//button[contains(text(),'Login')]")).click();
        Thread.sleep(3000);

        driver.get("https://safevoice-webapp.onrender.com/dashboard/resolver");

        List<WebElement> solutionBoxes = wait.until(
            ExpectedConditions.visibilityOfAllElementsLocatedBy(
                By.xpath("//textarea | //input[@type='text']")
            )
        );
        WebElement latestSolutionBox = solutionBoxes.get(0);
        latestSolutionBox.sendKeys(
            "Latest issue resolved successfully after proper investigation."
        );
        List<WebElement> submitButtons = driver.findElements(
            By.xpath("//button[contains(text(),'Submit')]")
        );
        WebElement latestSubmitButton = submitButtons.get(0);
        ((JavascriptExecutor) driver)
            .executeScript("arguments[0].scrollIntoView(true);", latestSubmitButton);
        ((JavascriptExecutor) driver)
            .executeScript("arguments[0].click();", latestSubmitButton);
        Thread.sleep(3000);
        Assert.assertTrue(driver.getCurrentUrl().contains("resolver"));
        System.out.println("Jagruti Tak 9111");
        System.out.println("IT-03 Passed: Resolver submitted solution for latest issue");
    }
    @AfterTest
    public void tearDown() {
        driver.quit(); }
}
```

Output :-

The screenshot displays the Eclipse IDE console and a web browser. The console shows the execution of a TestNG suite, resulting in a passed test. The web browser shows the 'Resolver Login' and 'Resolver Signup' forms, and a dashboard view of an assigned issue.

```
<terminated> IT03_ResolverSubmitSolution [TestNG] C:\Eclipse\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64
[RemoteTestNG] detected TestNG version 6.7.0
[TestNG] Running:
  C:\Users\Vishal\AppData\Local\Temp\testng-eclipse--352546058\testng-customsuite.xml

Jagruti Tak 9111
IT-03 Passed: Resolver submitted solution for latest issue
PASSED: resolverSubmitsSolutionForLatestIssue

=====
      Default test
      Tests run: 1, Failures: 0, Skips: 0
=====

=====
Default suite
Total tests run: 1, Failures: 0, Skips: 0
=====

[TestNG] Time taken by [TestListenerAdapter] Passed:0 Failed:0 Skipped:0]: 0 ms
[TestNG] Time taken by org.testng.reporters.jq.Main@6c372fe6: 71 ms
[TestNG] Time taken by org.testng.reporters.JUnitReportReporter@6fa34d52: 16 ms
[TestNG] Time taken by org.testng.reporters.XMLReporter@1ef6d34c: 18 ms
[TestNG] Time taken by org.testng.reporters.EmailableReporter@5495333e: 4 ms
[TestNG] Time taken by org.testng.reporters.SuiteHTMLReporter@135606db: 57 ms
```

The web browser shows the 'Resolver Authentication' page with a 'Back to Home' link. It contains two forms: 'Resolver Login' and 'Resolver Signup'. The 'Resolver Login' form has fields for 'Email' (Ji@gmail.com) and 'Password' (masked with dots), and a 'Login' button. The 'Resolver Signup' form has fields for 'Name', 'Select Role' (dropdown), 'Email', 'Phone', and 'Password', and a 'Signup' button.

The dashboard view shows the 'SafeVoice - Resolver' header and a 'Assigned Issues' section. The assigned issue details are:

- User: Jagruti
- Title: Test Issue
- Description: Integration testing for user issue reporting
- Category: Harassment
- Status: Resolved
- Solution: Latest issue resolved successfully after proper investigation.

The 'Update' button is visible at the bottom of the issue details.

The screenshot displays the TestNG results window for the class `IT03_ResolverSubmitSolution`. The top bar indicates 1 test passed, 0 failed, and 0 skipped, with a total time of 18420 ms. The test suite is expanded, showing the test `Default test (10.539 s)` and the method `resolverSubmitsSolutionForLatestIssue (10.539 s)`. The failure exception is empty, indicating a successful test. Below the test results, the Data Output tab is active, showing a table with 2 rows of data.

	solution_id [PK] integer	issue_id integer	resolver_id integer	solution_text
1	2	1	1	first of all move to trusted and safe place and go to near by police station to
2	3	4	1	Latest issue resolved successfully after proper investigation.

Using Selenium WebDriver and TestNG, integration testing validated the end-to-end workflow of the SafeVoice application:

- User reports an issue
- Admin assigns a resolver
- Resolver submits a solution

All integrated modules worked correctly, ensuring system reliability and functional accuracy.

3)System Testing :

System Testing is performed to verify the complete and integrated application as a whole in a real deployed environment.

It ensures that all modules work together correctly according to the system requirements.

This test validates the entire SafeVoice workflow from start to end:

- User reports an issue
- Admin assigns the issue
- Resolver submits a solution
- User acknowledges the solution

Since all roles, modules, and real application URLs are involved, this test is classified as System Testing.

What is Tested

1. User Module

- User login
- Issue reporting with title, description, and category
- Viewing submitted issues
- Acknowledging the resolver's solution

2. Admin Module

- Admin login
- Viewing reported issues
- Assigning issues to resolvers

3. Resolver Module

- Resolver login
- Viewing assigned issues
- Submitting solutions

4. Session Handling

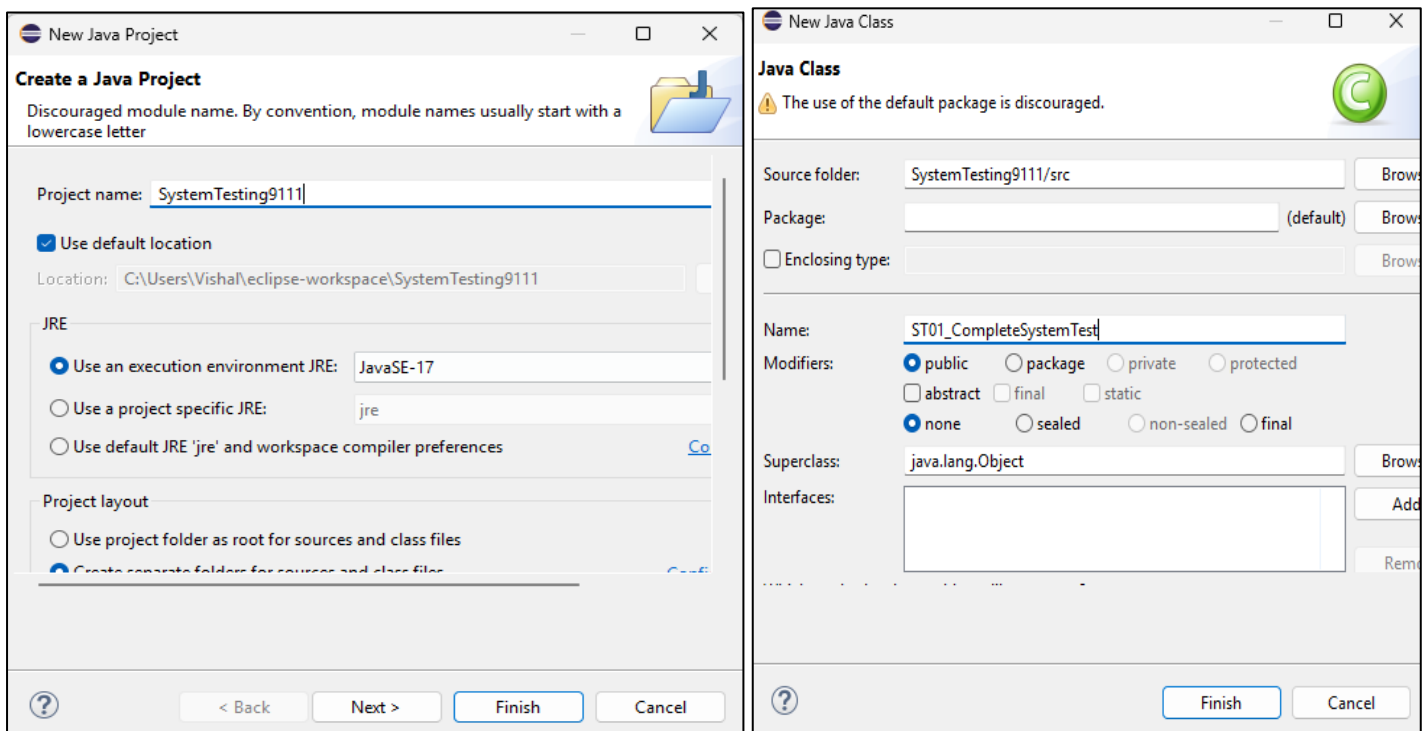
- Proper login and logout for all roles
- Secure navigation between sessions

Testing Approach

- Black Box Testing (testing without code-level knowledge)
- Automation Testing
- End-to-End Testing
- Real-Time Deployed System Testing

Steps1: Download selenium-server-standalone-3.141.59 jar and selenium-java-4.38 and download geckodriver . After installation make folder “myfolder” in that add all extracted selenium-java jar files and standalone jar, jcommder jar , testing jar , jxl jar and also add extracted geckodriver.exe file path inside java code.

Step2 :-Open Eclipse → Create a new java Project → Name it → Click on Finish. And Create a java Class → Right click on project → new → class → name it → Finish. And write below code in that and run that java class.



Code :

```
import org.openqa.selenium.*;
import org.openqa.selenium.firefox.*;
import org.openqa.selenium.support.ui.*;
import org.testng.Assert;
import org.testng.annotations.*;
import java.time.Duration;
import java.util.List;
public class ST01_CompleteSystemTest {
    WebDriver driver;
    WebDriverWait wait;
    @BeforeTest
```

```
public void setup() {
    System.setProperty("webdriver.gecko.driver", "D:\\myFolder\\geckodriver.exe");
    FirefoxOptions options = new FirefoxOptions();
    options.setBinary("C:\\Program Files\\Mozilla Firefox\\firefox.exe");
    driver = new FirefoxDriver(options);
    driver.manage().window().maximize();
    wait = new WebDriverWait(driver, Duration.ofSeconds(20));
}
public void safeLogout() throws InterruptedException {
    Thread.sleep(2000);
    WebElement logout = driver.findElement(By.linkText("Logout"));
    ((JavascriptExecutor) driver).executeScript("arguments[0].click();", logout);
    Thread.sleep(2000);
}
@Test
public void completeSystemFlow() throws InterruptedException {
    driver.get("https://safevoice-webapp.onrender.com/login/user");
    driver.findElement(By.name("email")).sendKeys("J@gmail.com");
    driver.findElement(By.name("password")).sendKeys("1234");
    driver.findElement(By.xpath("//button[contains(text(),'Login')]")).click();
    Thread.sleep(3000);
    driver.findElement(By.linkText("Report Issue")).click();
    Thread.sleep(2000);
    driver.findElement(By.name("title")).sendKeys("System Test Issue");
    driver.findElement(By.name("description"))
        .sendKeys("End-to-end system testing");
    new Select(driver.findElement(By.name("category")))
        .selectByVisibleText("Harassment");
    driver.findElement(By.xpath("//button[contains(text(),'Submit')]")).click();
    safeLogout();

    driver.get("https://safevoice-webapp.onrender.com/login/admin");
    driver.findElement(By.name("email")).sendKeys("a@gmail.com");
    driver.findElement(By.name("password")).sendKeys("421");
    driver.findElement(By.xpath("//button[contains(text(),'Login')]")).click();
    Thread.sleep(3000);
    driver.findElement(By.linkText("Assign Resolver")).click();
    wait.until(driver -> {
        List<WebElement> selects = driver.findElements(By.tagName("select"));
        return selects.size() >= 2 &&
            new Select(selects.get(0)).getOptions().size() > 1 &&
            new Select(selects.get(1)).getOptions().size() > 1;
    });
    List<WebElement> dropdowns = driver.findElements(By.tagName("select"));
    new Select(dropdowns.get(0)).selectByIndex(1);
    new Select(dropdowns.get(1)).selectByIndex(1);
    WebElement assignBtn =
        driver.findElement(By.xpath("//button[contains(text(),'Assign')]"));
}
```

```
((JavascriptExecutor) driver).executeScript("arguments[0].click();", assignBtn);
safeLogout();

driver.get("https://safevoice-webapp.onrender.com/login/resolver");
driver.findElement(By.name("email")).sendKeys("Ji@gmail.com");
driver.findElement(By.name("password")).sendKeys("789");
driver.findElement(By.xpath("//button[contains(text(),'Login')]")).click();
wait.until(ExpectedConditions.presenceOfElementLocated(By.xpath("//textarea")));
WebElement solutionBox =
    driver.findElements(By.xpath("//textarea")).get(0);
solutionBox.sendKeys("Issue resolved after investigation.");
WebElement submitBtn =
    driver.findElements(By.xpath("//button[contains(text(),'Submit')]")).get(0);
((JavascriptExecutor) driver).executeScript("arguments[0].click();", submitBtn);
safeLogout();

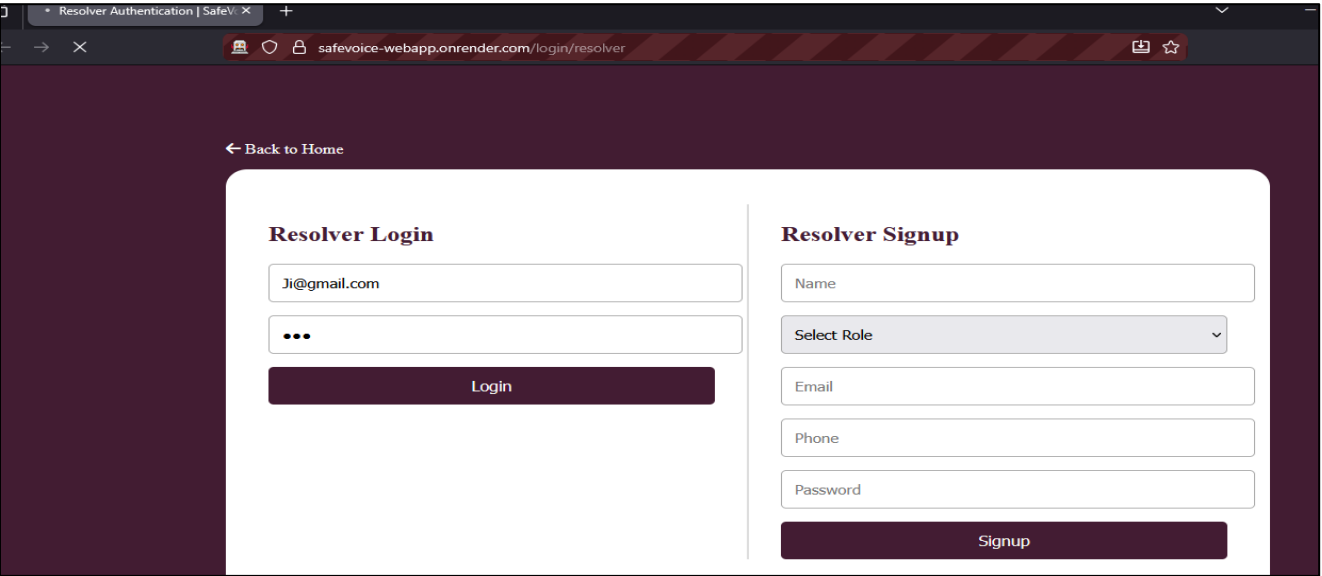
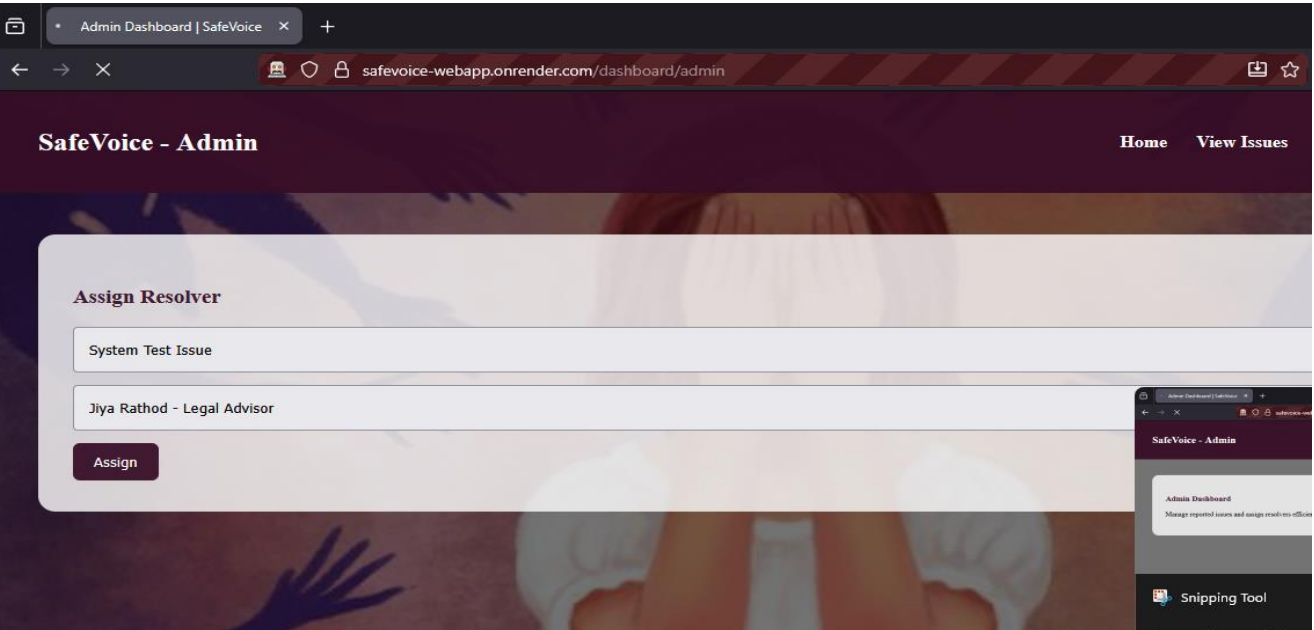
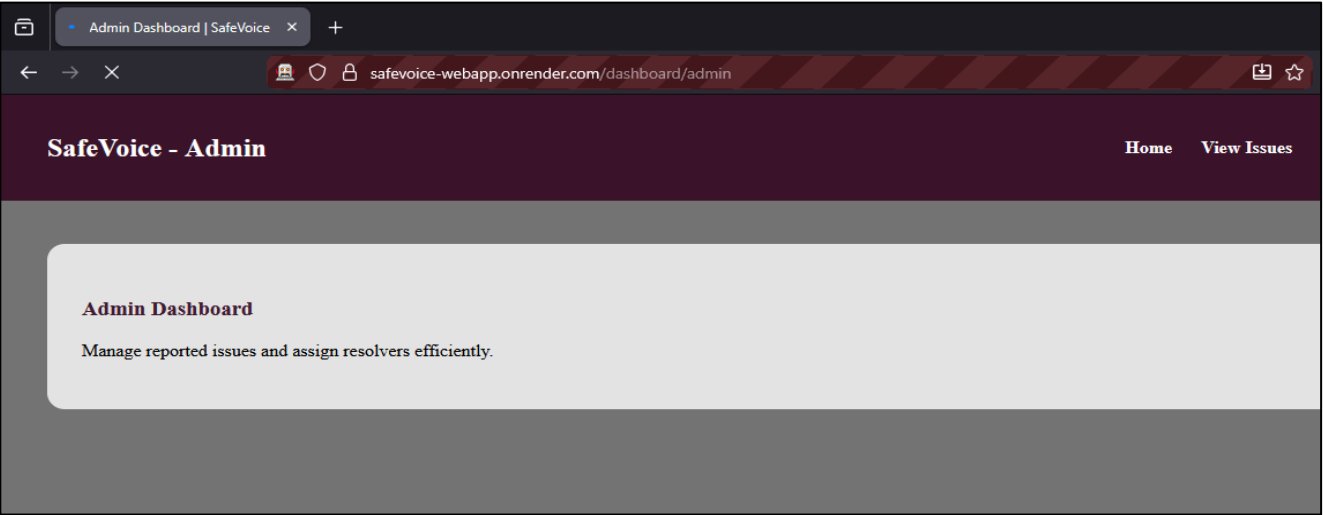
driver.get("https://safevoice-webapp.onrender.com/login/user");
driver.findElement(By.name("email")).sendKeys("J@gmail.com");
driver.findElement(By.name("password")).sendKeys("1234");
driver.findElement(By.xpath("//button[contains(text(),'Login')]")).click();
wait.until(ExpectedConditions.elementToBeClickable(
    By.linkText("My Issues")
)).click();
wait.until(ExpectedConditions.presenceOfElementLocated(
    By.xpath("//button[contains(text(),'I have read this solution')]")
));
WebElement acknowledgeBtn =
    driver.findElements(By.xpath("//button[contains(text(),'I have read this
solution')]")).get(0);
((JavascriptExecutor) driver)
    .executeScript("arguments[0].click();", acknowledgeBtn);
Thread.sleep(2000);
Assert.assertTrue(driver.getCurrentUrl().contains("user"));
System.out.println("Name : Jagruti Tak");
System.out.println("Roll No : 9111");
System.out.println("SYSTEM TEST PASSED");
System.out.println("User → Admin → Resolver → User Acknowledgement verified");
safeLogout();
}
@AfterTest
public void tearDown() {
    driver.quit();
}
}
```

Output :

The screenshot shows the 'User Authentication | SafeVoice' page. The browser address bar displays 'safevoice-webapp.onrender.com/login/user'. The page features a dark purple header with a '← Back to Home' link. The main content area is divided into two sections: 'User Login' and 'User Signup'. The 'User Login' section contains a text input field with 'J@gmail.com', a password input field with four dots, and a dark purple 'Login' button. The 'User Signup' section contains four text input fields labeled 'Name', 'Email', 'Phone', and 'Password', followed by a partially visible 'Sign' button.

The screenshot shows the 'User Dashboard | SafeVoice' page. The browser address bar displays 'safevoice-webapp.onrender.com/dashboard/user'. The page has a dark purple header with the 'SafeVoice' logo on the left and navigation links 'Home', 'Report Issue', 'My Issues', and 'Logout' on the right. Below the header is a large banner image of a group of people. A white card in the center contains the text 'Welcome to SafeVoice' and 'Report safety issues securely and track resolutions.'

The screenshot shows the 'Report an Issue' form on the 'User Dashboard | SafeVoice' page. The browser address bar displays 'safevoice-webapp.onrender.com/dashboard/user'. The form is a white card with a dark purple header 'Report an Issue'. It contains three text input fields with the following text: 'System Test Issue', 'End-to-end system testing', and 'Harassment'. At the bottom of the card is a dark purple 'Submit Issue' button.



Resolver Dashboard | SafeVoice

safevoice-webapp.onrender.com/dashboard/resolver?success=Solution submitted successfully

SafeVoice - Resolver

Assigned Issues Logout

Assigned Issues

User: Jagruti

Title: System Test Issue

Description: End-to-end system testing

Category: Harassment

Status: Resolved

Solution:

Issue resolved after investigation.

Issue resolved after investigation.

Update

User Authentication | SafeVoice

safevoice-webapp.onrender.com/login/user

← Back to Home

User Login

J@gmail.com

....

Login

User Signup

Name

Email

Phone

Password

Sign Up

User Dashboard | SafeVoice

safevoice-webapp.onrender.com/dashboard/user

SafeVoice

My Issues

Title: System Test Issue

Category: Harassment

Status: Resolved

Reported → Assigned → Resolved

✓ Verified Resolution

Solution:

Issue resolved after investigation.

I have read this solution

17	17	1	System Test Is...	End-to-end syste...	Harassment	Pen...	2026-01-23 14:25:50.568617	false
18	18	1	System Test Is...	End-to-end syste...	Harassment	Res...	2026-01-23 14:29:45.705186	true

```
@ Javadoc Declaration Console X
<terminated> ST01_CompleteSystemTest [TestNG] C:\Eclipse\workspace\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.1
[RemoteTestNG] detected TestNG version 6.7.0
[TestNG] Running:
  C:\Users\Vishal\AppData\Local\Temp\testng-eclipse--971491915\testng-customsuite.xml

Name : Jagruti Tak
Roll No : 9111
SYSTEM TEST PASSED
User → Admin → Resolver → User Acknowledgement verified
PASSED: completeSystemFlow

-----
Default test
Tests run: 1, Failures: 0, Skips: 0
-----

Default suite
Total tests run: 1, Failures: 0, Skips: 0
-----

[TestNG] Time taken by [TestListenerAdapter] Passed:0 Failed:0 Skipped:0]: 0 ms
[TestNG] Time taken by org.testng.reporters.jq.Main@6c372fe6: 32 ms
[TestNG] Time taken by org.testng.reporters.JUnitReportReporter@6fa34d52: 9 ms
[TestNG] Time taken by org.testng.reporters.XMLReporter@1ef6d34c: 10 ms
[TestNG] Time taken by org.testng.reporters.EmailableReporter@5495333e: 2 ms
[TestNG] Time taken by org.testng.reporters.SuiteHTMLReporter@135606db: 22 ms
```

4) Performance Testing :

Performance Testing is conducted to evaluate the responsiveness, stability, and reliability of the SafeVoice web application under normal load conditions. The objective is to ensure that the application performs efficiently when multiple users access it simultaneously.

Tool Used

- Apache JMeter

Reason:

Apache JMeter is a non-UI performance testing tool used to simulate multiple virtual users and measure server response time, throughput, and error rate without relying on browser-based interactions.

Install Apache JMeter

Go to browser → search apache JMeter download → click on that zip file to download it → extract that zip and in that click on bin folder → then click on JMeter batch file → it will open apache JMeter interface to perform the load testing.

Apache JMeter 5.6.3 (Requires Java 8+)

Binaries
[apache-jmeter-5.6.3.tgz sha512 pgp](#)
[apache-jmeter-5.6.3.zip sha512 pgp](#)

Today

apache-jmeter-5.6.3	23-01-2026 20:56
apache-jmeter-5.6.3	23-01-2026 21:17

printable_docs	23-01-2026 21:17	File folder
docs	23-01-2026 21:17	File folder
extras	23-01-2026 21:17	File folder
lib	23-01-2026 21:17	File folder
bin	23-01-2026 21:17	File folder

jaas.conf	23-01-2026 21:17	CONF File	2 KB
jmeter	23-01-2026 21:17	File	9 KB
jmeter	23-01-2026 21:17	Windows Batch File	9 KB
jmeter	23-01-2026 21:17	Properties Source ...	58 KB
jmeter.sh	23-01-2026 21:17	sh_auto_file	5 KB
jmeter.n	23-01-2026 21:17	Windows Comma	2 KB

Apache JMeter (5.6.3)

File Edit Search Run Options Tools Help

Test Plan

C:\WINDOWS\system32\cmd. X + v

```
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release

=====
Don't use GUI mode for load testing !, only for Test creation and Test debugging.
For load testing, use CLI Mode (was NON GUI):
jmeter -n -t [jmx file] -l [results file] -e -o [Path to web report folder]
& increase Java Heap to meet your test requirements:
Modify current env variable HEAP="-Xms1g -Xmx1g -XX:MaxMetaspaceSize=256m" in the jmeter batch file
Check : https://jmeter.apache.org/usermanual/best-practices.html
=====
```

1) Login + Report Issue Performance Testing

(50 users in 10 seconds)

In this test, 50 virtual users simultaneously performed the following actions within 10 seconds:

- User login
- Report issue submission

Observation

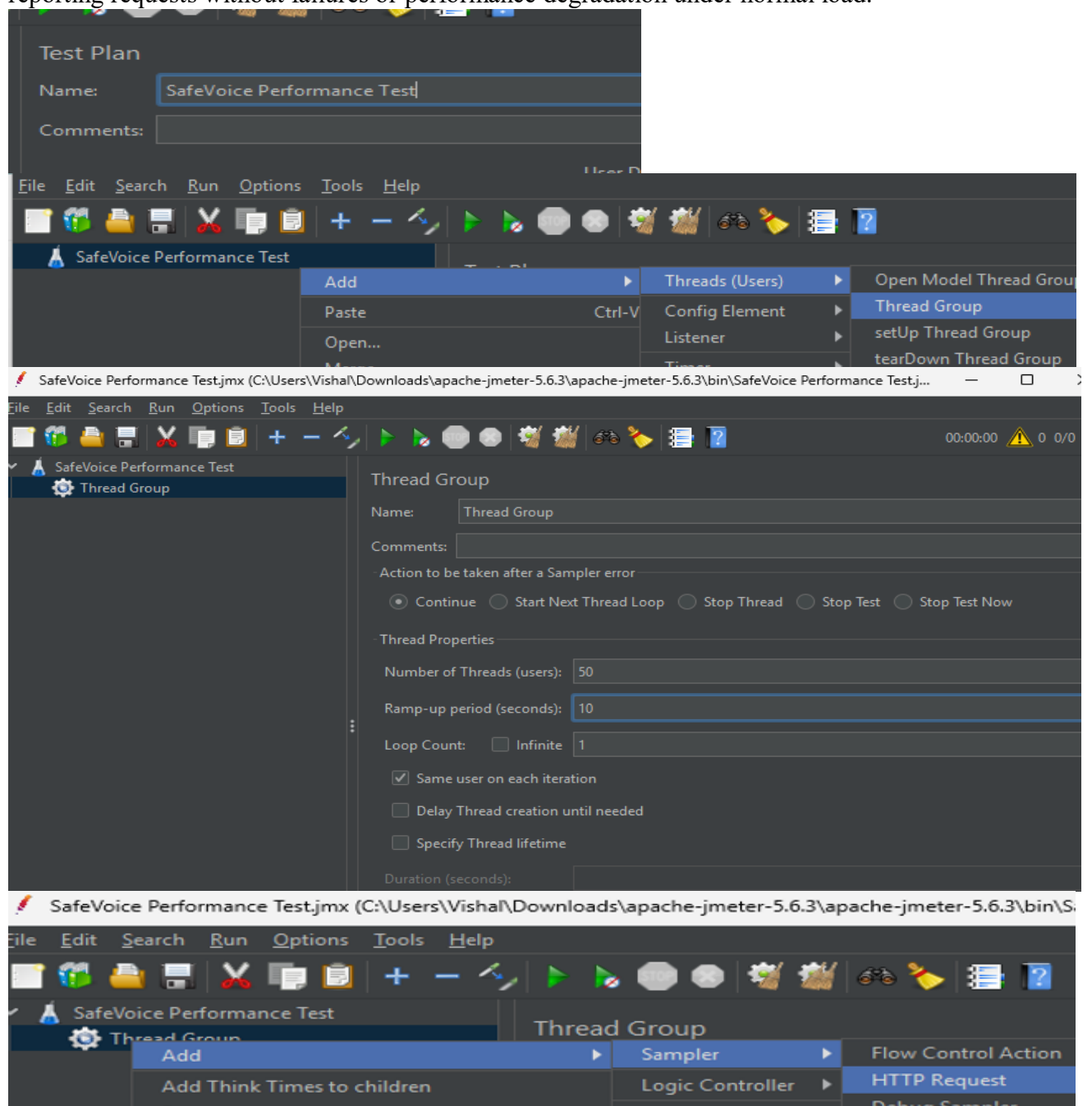
- The server returned HTTP 307 Temporary Redirect
- Error Count: 0

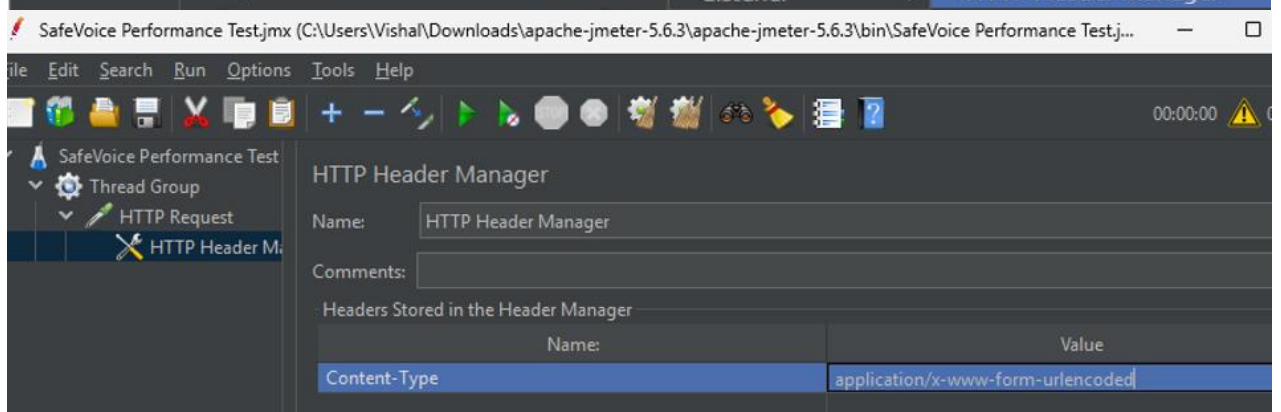
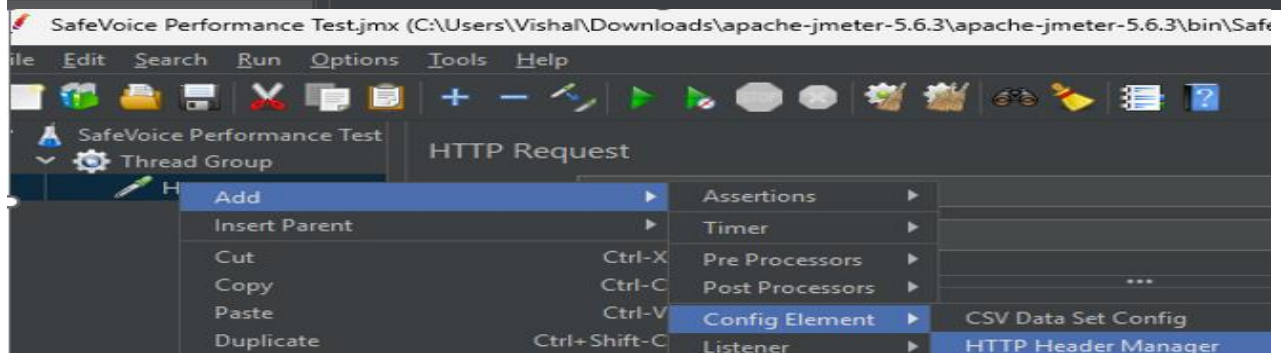
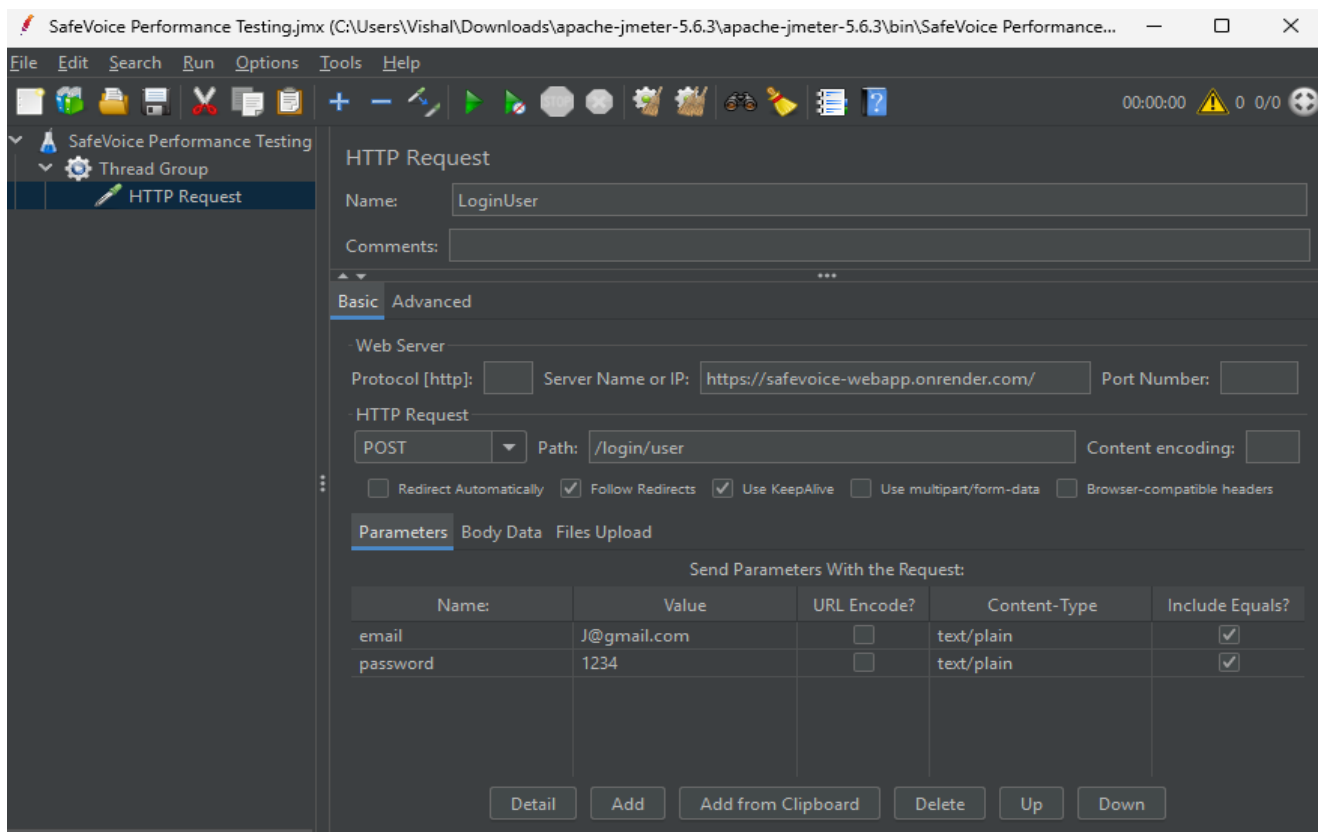
Explanation

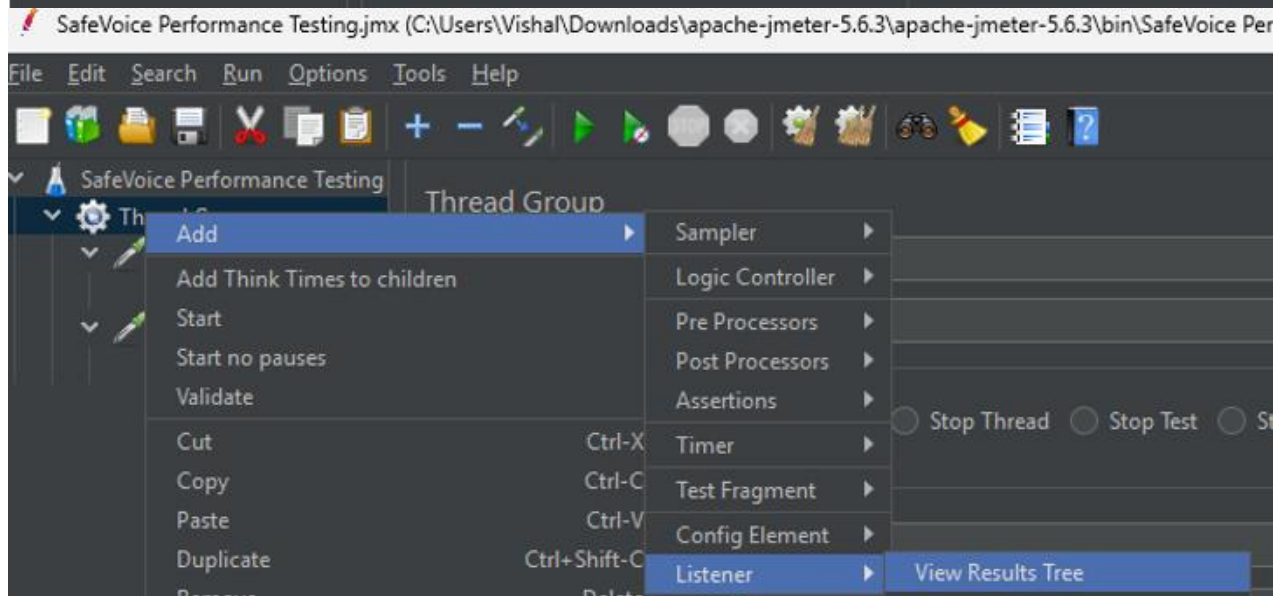
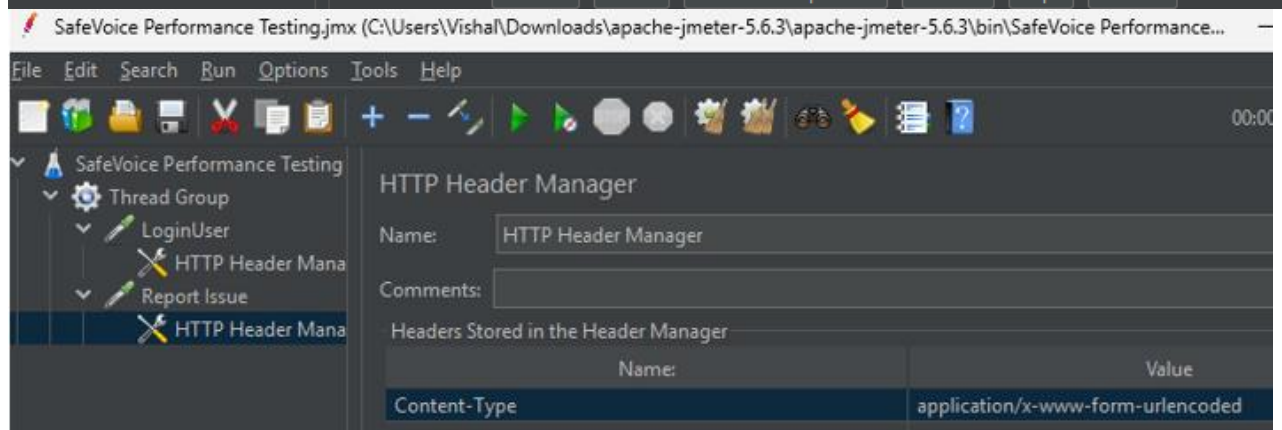
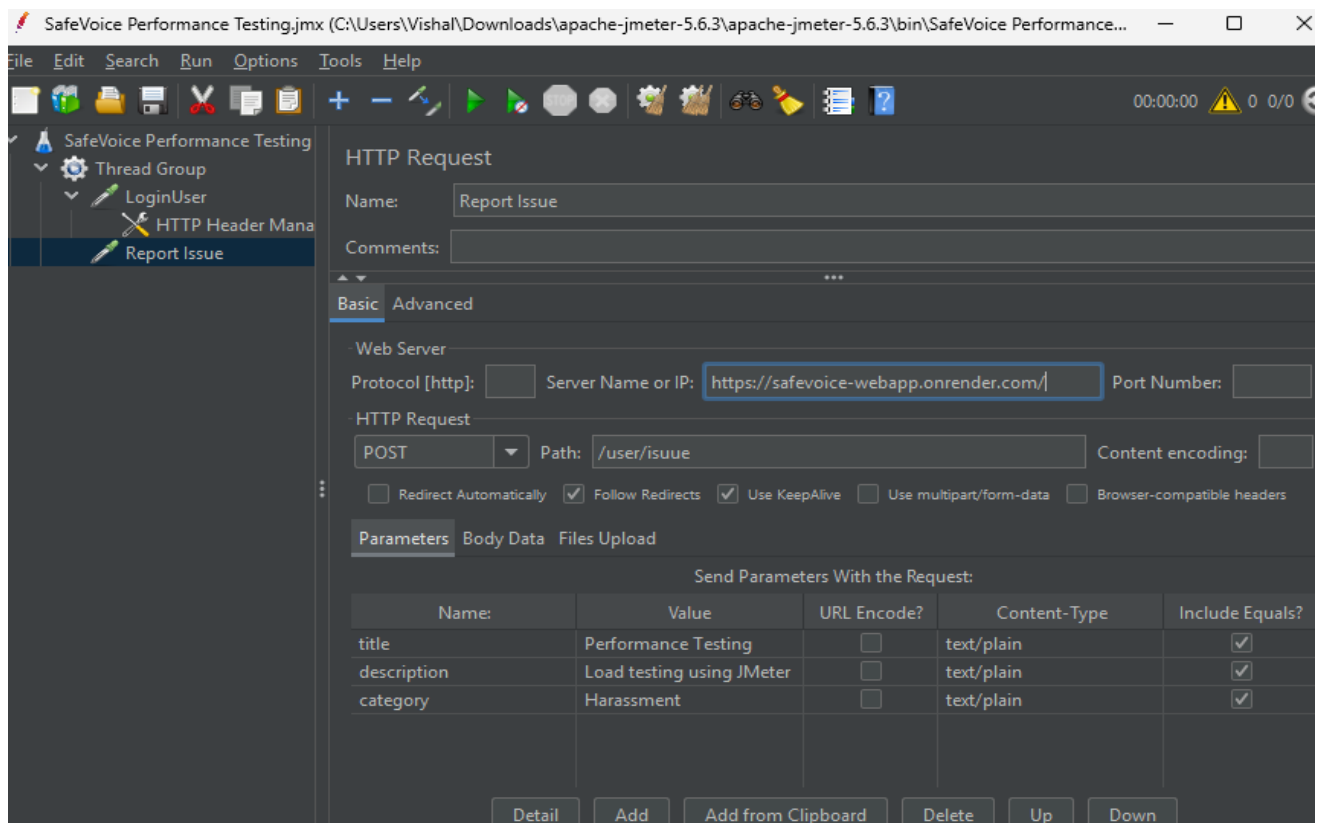
The HTTP 307 Temporary Redirect indicates that:

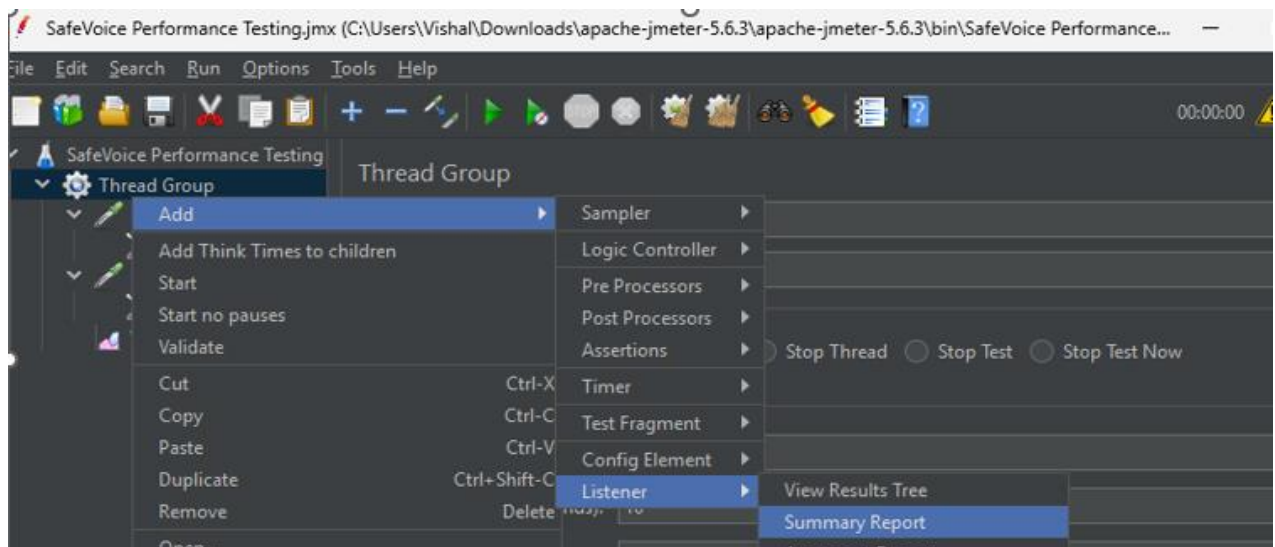
- Login credentials were successfully validated
- The server redirected the user to the dashboard after authentication
- Since JMeter does not follow UI navigation, the redirect confirms a successful login

This confirms that the SafeVoice application can efficiently handle concurrent login and issue reporting requests without failures or performance degradation under normal load.

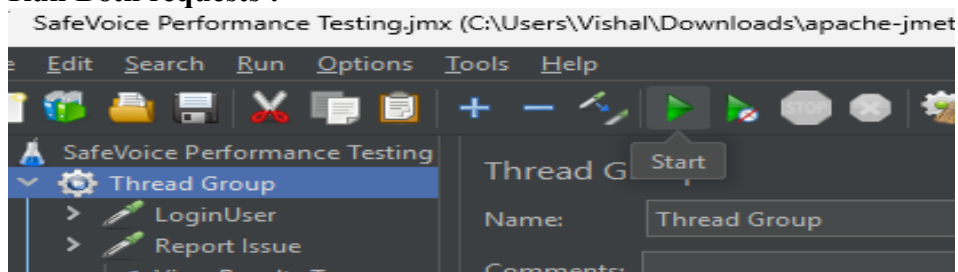




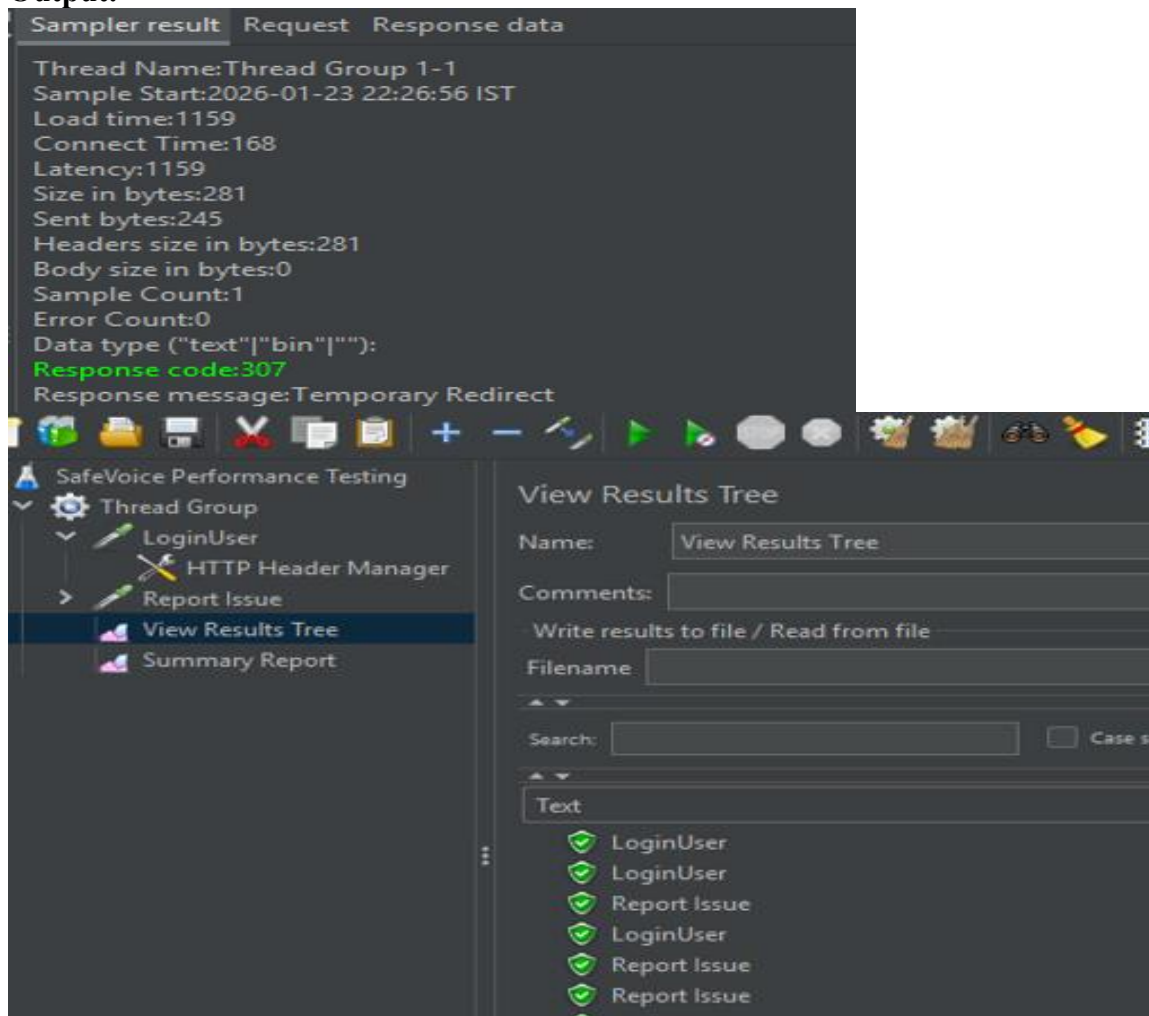




Run Both requests :



Output:



Summary Report

Name:

Summary Report

Comments:

Write results to file / Read from file

Filename

Browse...

Log/Display Only: ☐ Errors ☐ Successes

Configure

Label ↑	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
LoginUser	50	464	314	803	116.07	0.00%	4.9/sec	1.35	1.17	281.0
Report Issue	50	328	260	634	68.09	0.00%	4.9/sec	1.35	1.42	281.0
TOTAL	100	396	260	803	116.96	0.00%	9.5/sec	2.61	2.51	281.0

2) Stress Testing on Login + Report Issue (100 users in 10 seconds)

Stress testing was performed by increasing the load beyond normal conditions. In this test, 100 users hit the server within 10 seconds, performing login and report issue operations.

Purpose

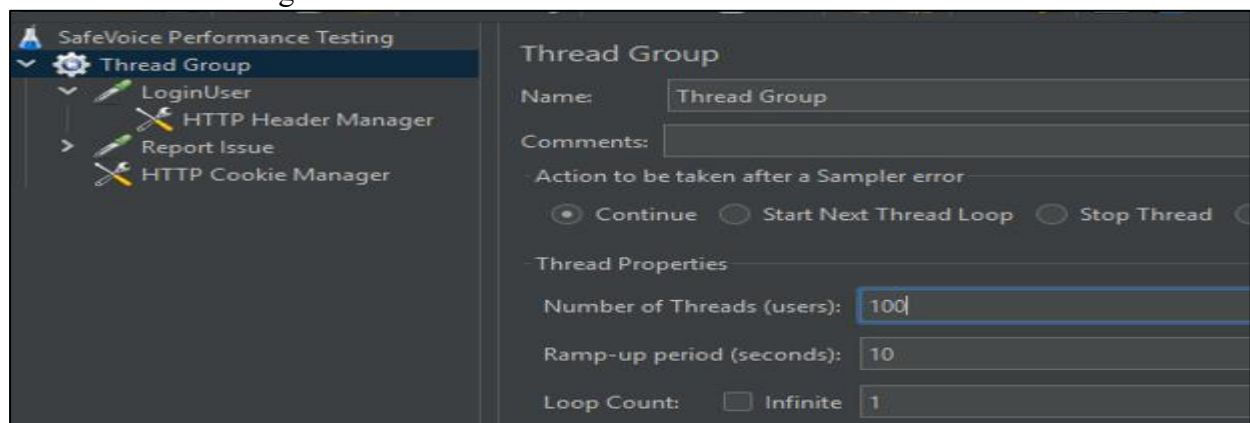
- To determine the application's behavior under extreme load
- To identify performance limits and server stability

Observation

- Server continued responding without crashing
- No authentication failures occurred
- Redirect responses confirmed successful processing

Explanation

This test verifies that SafeVoice can tolerate sudden spikes in user activity. The application maintained stability even under heavy load, indicating proper backend handling and efficient server resource management.



Summary Report

Name:

Summary Report

Comments:

Write results to file / Read from file

Filename

Browse...

Log/Display Only:

☐ Errors

☐ Successes

Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
LoginUser	100	498	290	2097	309.10	0.00%	8.6/sec	2.35	2.05	281.0
Report Issue	100	331	266	942	92.94	0.00%	9.3/sec	2.56	2.69	281.0
TOTAL	200	415	266	2097	243.09	0.00%	16.5/sec	4.54	4.36	281.0

Comparing both test results :

Parameter	Performance Testing (50 users / 10 sec)	Stress Testing (100 users / 10 sec)
No. of Users	50 users	100 users
Total Samples	100	200
Avg Response Time	~396 ms	~415 ms
Max Response Time	803 ms	2097 ms
Throughput	~9.5 req/sec	~16.5 req/sec
Error %	0.00%	0.00%
Stability	Stable	Stable under heavy load

Performance Testing (Without Stress)

- 50 users performed Login + Report Issue within 10 seconds.
- Response time was fast and consistent.
- No errors occurred.
- The system handled normal load smoothly.

Result: Application performs well under expected user load.

Stress Testing

- 100 users hit the server in the same 10 seconds.
- Response time increased, especially the maximum response time.
- Throughput increased, showing the server handled more requests.
- Still 0% error rate, meaning no failures.

Result: Application remains stable even under heavy load, though response time increases.

Conclusion

- SafeVoice performs efficiently under normal usage.
- Under stress, the system slows down slightly but does not crash or fail.
- This proves the application is reliable, scalable, and stable.

5) UI Testing (whole system UI Testing) :

UI (User Interface) Testing is a type of software testing that verifies whether all visual elements of a web application are present, visible, and working correctly according to requirements. It focuses on how the application looks and how users interact with it, rather than internal logic.

The main objective of UI testing in the SafeVoice application is to ensure that all user-facing **components** across User, Admin, and Resolver modules are:

- Properly displayed
- Correctly structured
- Easily accessible
- Consistent across pages

This helps improve usability, accessibility, and user experience.

Tool Used for UI Testing

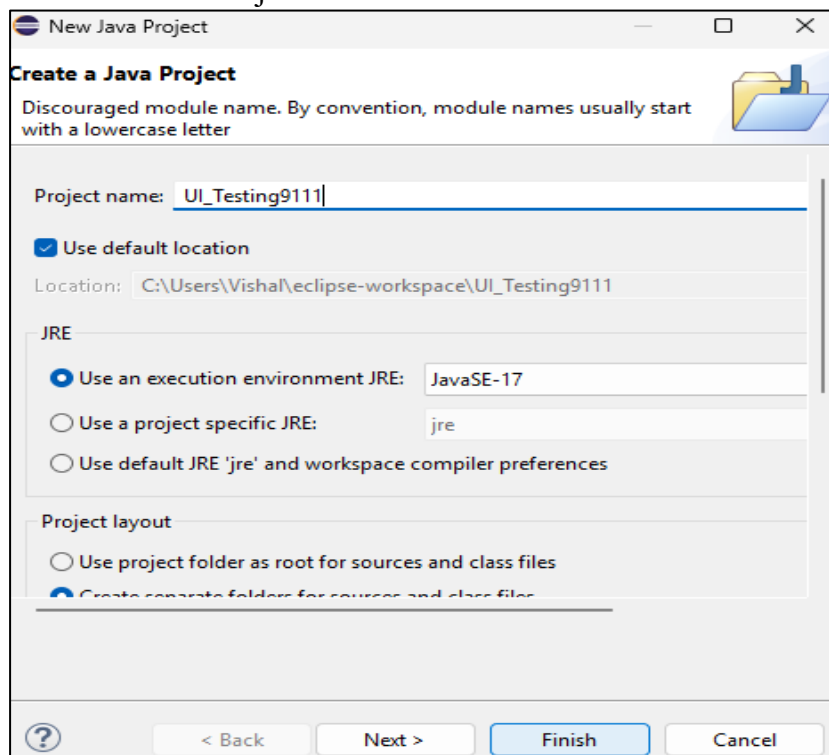
- Selenium WebDriver
- TestNG
- Mozilla Firefox (GeckoDriver)

How UI Testing Was Performed

1. The application was launched in a real browser (Firefox).
2. Each major page was visited:
 - User Login
 - User Dashboard
 - Report Issue Page
 - Admin Dashboard
 - Resolver Dashboard
 - My Issues Page
3. For every page:
 - UI elements were counted using HTML tags
 - Visibility of critical elements (email, password, buttons, dropdowns) was asserted
 - Navigation menus and forms were verified
4. End-to-end UI flow was validated:
 - User → Admin → Resolver → User acknowledgement

Steps1: Download selenium-server-standalone-3.141.59 jar and selenium-java-4.38 and download geckodriver . After installation make folder “myfolder” in that add all extracted selenium-java jar files and standalone jar, jcommder jar , testing jar , jxl jar and also add extracted geckodriver.exe file path inside java code.

Step2 :-Open Eclipse → Create a new java Project → Name it → Click on Finish. And Create a java Class → Right click on project → new → class → name it → Finish. And write below code in that and run that java class.



Code :

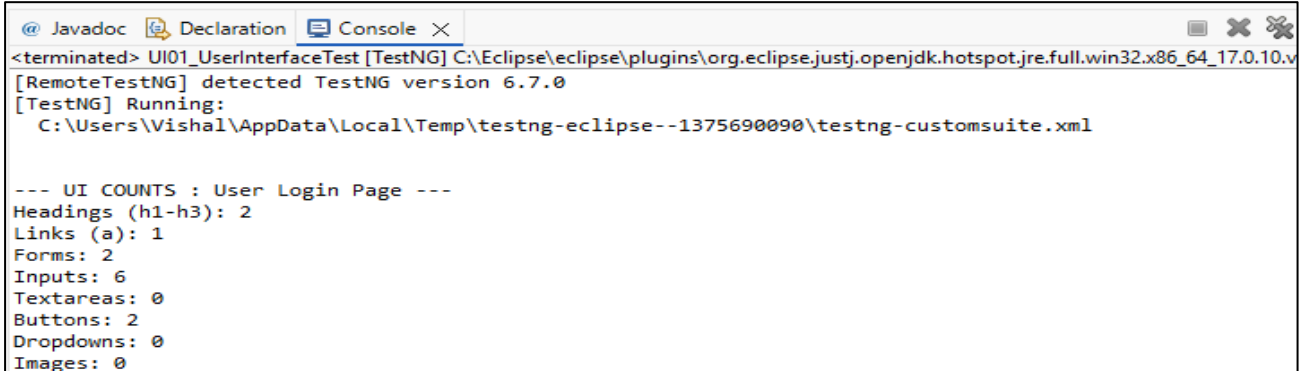
```
import org.openqa.selenium.*;  
import org.openqa.selenium.firefox.*;  
import org.openqa.selenium.support.ui.*;  
import org.testng.Assert;
```

```
import org.testng.annotations.*;
import java.time.Duration;
import java.util.List;
public class UI01_UserInterfaceTest {
    WebDriver driver; WebDriverWait wait;
    @BeforeTest
    public void setup() {
        System.setProperty("webdriver.gecko.driver", "D:\\myFolder\\geckodriver.exe");
        FirefoxOptions options = new FirefoxOptions();
        options.setBinary("C:\\Program Files\\Mozilla Firefox\\firefox.exe");
        driver = new FirefoxDriver(options);
        driver.manage().window().maximize();
        wait = new WebDriverWait(driver, Duration.ofSeconds(20));
    }
    public void safeLogout() throws InterruptedException {
        Thread.sleep(1500);
        WebElement logout = driver.findElement(By.linkText("Logout"));
        ((JavascriptExecutor) driver).executeScript("arguments[0].click();", logout);
        Thread.sleep(1500);
    }
    public void printUICounts(String pageName) {
        System.out.println("\n--- UI COUNTS : " + pageName + " ---");
        System.out.println("Headings (h1-h3): " +
            driver.findElements(By.cssSelector("h1,h2,h3")).size());
        System.out.println("Links (a): " +
            driver.findElements(By.tagName("a")).size());
        System.out.println("Forms: " +
            driver.findElements(By.tagName("form")).size());
        System.out.println("Inputs: " +
            driver.findElements(By.tagName("input")).size());
        System.out.println("Textareas: " +
            driver.findElements(By.tagName("textarea")).size());
        System.out.println("Buttons: " +
            driver.findElements(By.tagName("button")).size());
        System.out.println("Dropdowns: " +
            driver.findElements(By.tagName("select")).size());
        System.out.println("Images: " +
            driver.findElements(By.tagName("img")).size());
    }
    @Test
    public void completeApplication_UITesting() throws InterruptedException {
        driver.get("https://safevoice-webapp.onrender.com/login/user");
        printUICounts("User Login Page");
        Assert.assertTrue(driver.findElement(By.name("email")).isDisplayed());
        Assert.assertTrue(driver.findElement(By.name("password")).isDisplayed());
        driver.findElement(By.name("email")).sendKeys("J@gmail.com");
        driver.findElement(By.name("password")).sendKeys("1234");
        driver.findElement(By.tagName("button")).click();
        Thread.sleep(3000);
        printUICounts("User Dashboard");
        List<WebElement> navItems =
            driver.findElements(By.cssSelector(".navbar ul li"));
    }
}
```

```
System.out.println("Navbar menu items: " + navItems.size());
Assert.assertTrue(navItems.size() >= 3);
driver.findElement(By.linkText("Report Issue")).click();
Thread.sleep(2000);
printUICounts("Report Issue Page");
Select category =
    new Select(driver.findElement(By.name("category")));
System.out.println("Category dropdown options: " +
    category.getOptions().size());
Assert.assertTrue(category.getOptions().size() >= 3);
driver.findElement(By.name("title")).sendKeys("UI Testing Issue");
driver.findElement(By.name("description"))
    .sendKeys("Complete UI testing using Selenium");
category.selectByIndex(1);
driver.findElement(By.xpath("//button[contains(text(),'Submit')]")).click();
safeLogout();
driver.get("https://safevoice-webapp.onrender.com/login/admin");
printUICounts("Admin Login Page");
driver.findElement(By.name("email")).sendKeys("a@gmail.com");
driver.findElement(By.name("password")).sendKeys("421");
driver.findElement(By.tagName("button")).click();
Thread.sleep(3000);
printUICounts("Admin Dashboard");
driver.findElement(By.linkText("Assign Resolver")).click();
List<WebElement> selects =
    driver.findElements(By.tagName("select"));
Assert.assertTrue(selects.size() >= 2);
new Select(selects.get(0)).selectByIndex(1);
new Select(selects.get(1)).selectByIndex(1);
driver.findElement(By.xpath("//button[contains(text(),'Assign')]")).click();
safeLogout();
driver.get("https://safevoice-webapp.onrender.com/login/resolver");
printUICounts("Resolver Login Page");
driver.findElement(By.name("email")).sendKeys("Ji@gmail.com");
driver.findElement(By.name("password")).sendKeys("789");
driver.findElement(By.tagName("button")).click();
wait.until(ExpectedConditions.presenceOfElementLocated(By.tagName("textarea")));
printUICounts("Resolver Dashboard");
driver.findElement(By.tagName("textarea"))
    .sendKeys("Issue resolved after UI verification");
driver.findElement(By.xpath("//button[contains(text(),'Submit')]")).click();
safeLogout();
driver.get("https://safevoice-webapp.onrender.com/login/user");
driver.findElement(By.name("email")).sendKeys("J@gmail.com");
driver.findElement(By.name("password")).sendKeys("1234");
driver.findElement(By.tagName("button")).click();
wait.until(ExpectedConditions.elementToBeClickable(
    By.linkText("My Issues"))).click();
printUICounts("My Issues Page");
WebElement ackBtn =
    wait.until(ExpectedConditions.elementToBeClickable(
        By.xpath("//button[contains(text(),'I have read this solution')]")));
```

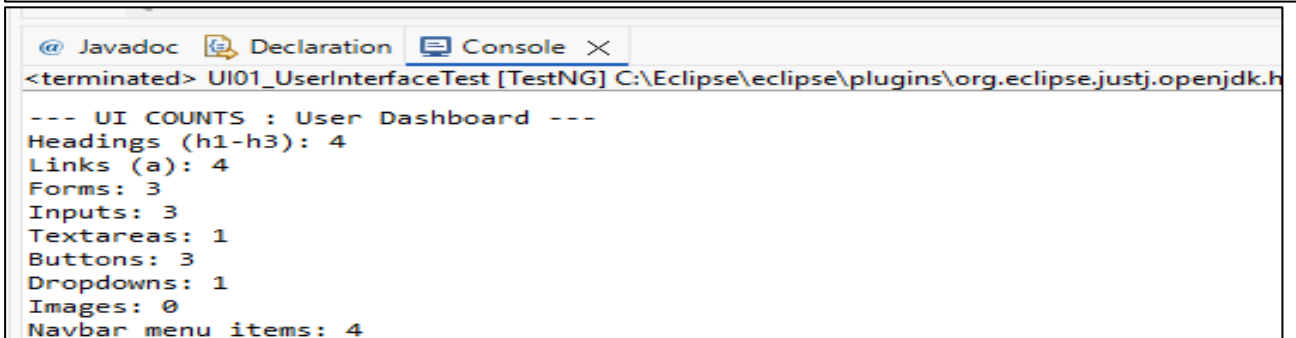
```
Assert.assertTrue(ackBtn.isDisplayed());  
ackBtn.click();  
System.out.println("\n COMPLETE UI TESTING PASSED");  
System.out.println("Name : Jagruti Tak");  
System.out.println("Roll No : 9111");  
safeLogout();  
}  
@AfterTest  
public void tearDown() {driver.quit();  }}
```

Output :



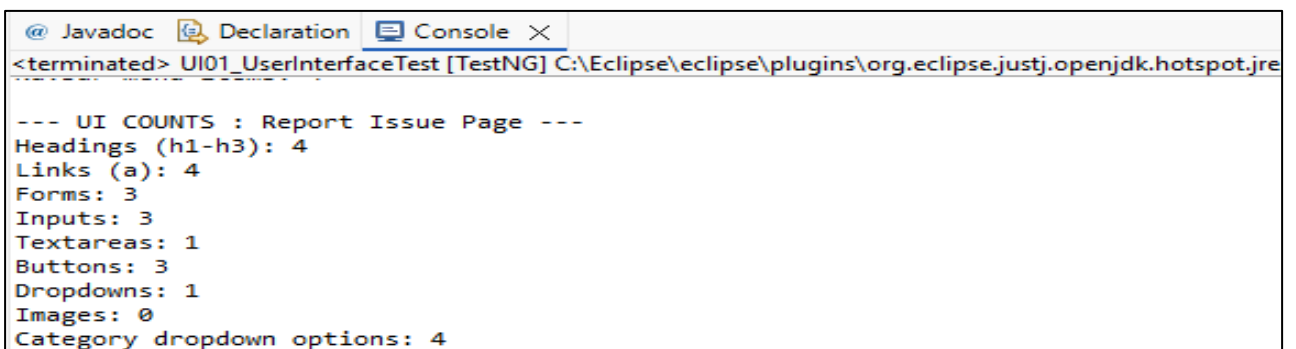
The screenshot shows the Eclipse IDE console with the 'Console' tab selected. It displays the output of a TestNG test run for 'UI01_UserInterfaceTest'. The output includes the TestNG version (6.7.0), the test suite file path, and a summary of UI counts for the 'User Login Page'. The counts are: Headings (h1-h3): 2, Links (a): 1, Forms: 2, Inputs: 6, Textareas: 0, Buttons: 2, Dropdowns: 0, and Images: 0.

```
<terminated> UI01_UserInterfaceTest [TestNG] C:\Eclipse\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.10.v...  
[RemoteTestNG] detected TestNG version 6.7.0  
[TestNG] Running:  
  C:\Users\Vishal\AppData\Local\Temp\testng-eclipse--1375690090\testng-customsuite.xml  
  
--- UI COUNTS : User Login Page ---  
Headings (h1-h3): 2  
Links (a): 1  
Forms: 2  
Inputs: 6  
Textareas: 0  
Buttons: 2  
Dropdowns: 0  
Images: 0
```



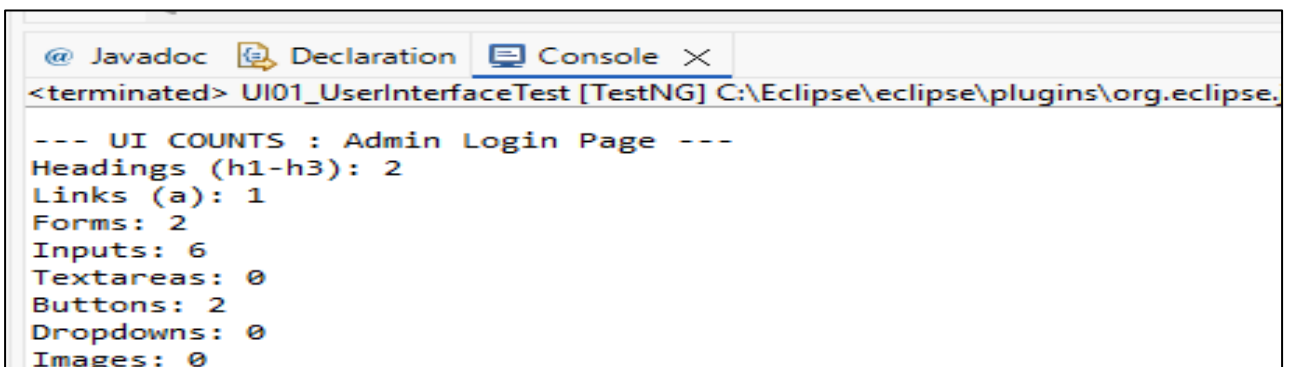
The screenshot shows the Eclipse IDE console with the 'Console' tab selected. It displays the output of a TestNG test run for 'UI01_UserInterfaceTest'. The output includes the TestNG version (6.7.0), the test suite file path, and a summary of UI counts for the 'User Dashboard'. The counts are: Headings (h1-h3): 4, Links (a): 4, Forms: 3, Inputs: 3, Textareas: 1, Buttons: 3, Dropdowns: 1, Images: 0, and Navbar menu items: 4.

```
<terminated> UI01_UserInterfaceTest [TestNG] C:\Eclipse\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre...  
  
--- UI COUNTS : User Dashboard ---  
Headings (h1-h3): 4  
Links (a): 4  
Forms: 3  
Inputs: 3  
Textareas: 1  
Buttons: 3  
Dropdowns: 1  
Images: 0  
Navbar menu items: 4
```



The screenshot shows the Eclipse IDE console with the 'Console' tab selected. It displays the output of a TestNG test run for 'UI01_UserInterfaceTest'. The output includes the TestNG version (6.7.0), the test suite file path, and a summary of UI counts for the 'Report Issue Page'. The counts are: Headings (h1-h3): 4, Links (a): 4, Forms: 3, Inputs: 3, Textareas: 1, Buttons: 3, Dropdowns: 1, Images: 0, and Category dropdown options: 4.

```
<terminated> UI01_UserInterfaceTest [TestNG] C:\Eclipse\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre...  
  
--- UI COUNTS : Report Issue Page ---  
Headings (h1-h3): 4  
Links (a): 4  
Forms: 3  
Inputs: 3  
Textareas: 1  
Buttons: 3  
Dropdowns: 1  
Images: 0  
Category dropdown options: 4
```



The screenshot shows the Eclipse IDE console with the 'Console' tab selected. It displays the output of a TestNG test run for 'UI01_UserInterfaceTest'. The output includes the TestNG version (6.7.0), the test suite file path, and a summary of UI counts for the 'Admin Login Page'. The counts are: Headings (h1-h3): 2, Links (a): 1, Forms: 2, Inputs: 6, Textareas: 0, Buttons: 2, Dropdowns: 0, and Images: 0.

```
<terminated> UI01_UserInterfaceTest [TestNG] C:\Eclipse\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre...  
  
--- UI COUNTS : Admin Login Page ---  
Headings (h1-h3): 2  
Links (a): 1  
Forms: 2  
Inputs: 6  
Textareas: 0  
Buttons: 2  
Dropdowns: 0  
Images: 0
```

```
@ Javadoc Declaration Console X
<terminated> UI01_UserInterfaceTest [TestNG] C:\Eclipse\eclipse\plugins\org.eclipse.j
--- UI COUNTS : Admin Dashboard ---
Headings (h1-h3): 4
Links (a): 4
Forms: 1
Inputs: 0
Textareas: 0
Buttons: 1
Dropdowns: 2
Images: 0
```

```
@ Javadoc Declaration Console X
<terminated> UI01_UserInterfaceTest [TestNG] C:\Eclipse\eclipse\plugins\org.eclipse.just
--- UI COUNTS : Resolver Login Page ---
Headings (h1-h3): 2
Links (a): 1
Forms: 2
Inputs: 6
Textareas: 0
Buttons: 2
Dropdowns: 1
Images: 0
```

```
@ Javadoc Declaration Console X
<terminated> UI01_UserInterfaceTest [TestNG] C:\Eclipse\eclipse\plugins\org.eclipse.justj.openjdk
--- UI COUNTS : Resolver Dashboard ---
Headings (h1-h3): 2
Links (a): 2
Forms: 24
Inputs: 44
Textareas: 14
Buttons: 24
Dropdowns: 0
Images: 0
```

```
@ Javadoc Declaration Console X
<terminated> UI01_UserInterfaceTest [TestNG] C:\Eclipse\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.
--- UI COUNTS : My Issues Page ---
Headings (h1-h3): 4
Links (a): 4
Forms: 4
Inputs: 4
Textareas: 1
Buttons: 4
Dropdowns: 1
Images: 0

UI Testing done successfully
Name : Jagruti Tak
Roll No : 9111
PASSED: completeApplication_UITesting

=====
      Default test
      Tests run: 1, Failures: 0, Skips: 0
=====

=====
Default suite
Total tests run: 1, Failures: 0, Skips: 0
=====

[TestNG] Time taken by [TestListenerAdapter] Passed:0 Failed:0 Skipped:0]: 0 ms
[TestNG] Time taken by org.testng.reporters.jq.Main@6c372fe6: 32 ms
[TestNG] Time taken by org.testng.reporters.XmlReporter@66f34f5a: 0 ms
```

6) Acceptance testing :

Acceptance Testing Approach

- **Type:** User Acceptance Testing (UAT)
- **Testing Method:** Black-Box Testing
- **Environment:** Live Deployed Application (Render Cloud)
- **Testing Basis:** Functional requirements and expected user behavior
- **Tester Role:** End-user simulation (User, Admin, Resolver)

Acceptance Criteria

The SafeVoice application is considered acceptable if:

- Users can successfully sign up and log in
- Users can report safety issues with correct details
- Admins can view and assign issues to resolvers
- Resolvers can submit solutions for assigned issues
- Users can view and acknowledge solutions
- UI components are clearly visible and usable
- The system handles multiple users without errors
- No critical failures occur during normal and peak load

Scenario ID	Acceptance Scenario	Expected Outcome
AT-01	User login and issue reporting	Issue submitted successfully
AT-02	Admin login and resolver assignment	Issue assigned correctly
AT-03	Resolver login and solution submission	Solution saved successfully
AT-04	User views and acknowledges solution	Issue marked as acknowledged
AT-05	Concurrent user access	No system crash or failure
AT-06	UI usability and navigation	Smooth and user-friendly experience

Note:

No separate screenshots or demonstrations are included for Acceptance Testing, as all major functional flows and outcomes were already validated and documented during Unit, Integration, System, Performance, and UI Testing phases. Acceptance Testing focused on final user validation and requirement compliance rather than repeated execution evidence.

Result:

SafeVoice application is accepted and ready for real-world usage.

Source Code :-

Safe Voice Web-application Source code

Source code of application is compressed into zip file :-

<https://drive.google.com/file/d/1FvjJX4FG8DD9W5Uj-9UEmQDRTvXtCznu/view?usp=sharing>

SWOT Analysis :

Strengths

- Role-based access control (User, Admin, Resolver)
- Fully deployed and tested on a live cloud platform
- End-to-end issue lifecycle management
- Secure authentication mechanism
- Stable performance under load and stress
- Comprehensive testing coverage (Unit → Acceptance)

Weaknesses

- No built-in mobile application (web-only access)
- Limited advanced security features like multi-factor authentication
- UI design can be further enhanced for accessibility
- JSON-based API support not implemented in some modules

Opportunities

- Integration with mobile applications
- Adding real-time notifications (SMS / Email)
- AI-based issue categorization and prioritization
- Government or NGO collaboration for wider reach
- Advanced analytics and reporting dashboards

Threats

- Cybersecurity risks if not continuously monitored
- Dependence on internet connectivity
- High traffic during emergency situations
- Data privacy regulations and compliance challenges

Conclusion

This project successfully demonstrated comprehensive software testing of the SafeVoice Women Safety Issue Reporting Web Application.

Multiple testing techniques including Unit Testing, Integration Testing, System Testing, Performance Testing, UI Testing, and Acceptance Testing were applied systematically.

Testing was performed on a real deployed cloud environment, which ensured accurate validation under real-world conditions.

The results confirmed that SafeVoice is functionally correct, user-friendly, stable under load, and reliable.

Through this project, practical experience was gained in:

- Automation testing using Selenium WebDriver and TestNG
- API testing using Postman
- Performance and stress testing using Apache JMeter
- End-to-end system validation on live applications

Overall, the SafeVoice application meets all quality standards and is ready for production use, fulfilling its objective of providing a secure platform for women safety issue reporting.